

**Universidad
Rey Juan Carlos**

INGENIERÍA TÉCNICA EN INFORMÁTICA DE SISTEMAS

Curso Académico 2002 / 2003

Proyecto de Fin de Carrera

**APLICACIÓN DEL FILTRO DE KALMAN AL
SEGUIMIENTO DE OBJETOS EN SECUENCIAS
DE IMÁGENES.**

Autor: Patricia Rodríguez Muñoz.

Tutores: Antonio Sanz Montemayor.

Juan José Pantrigo Fernández.

AGRADECIMIENTOS

A Antonio Sanz y Juan José Pantrigo por su paciencia y ayuda.

A Eva por tantas horas compartidas.

A José María Caverio y al grupo Kybele por prestarnos su equipo.

A Paul Y. Oh. por su aplicación en tiempo real.

A José Luis por prestarnos la web-cam.

A Nacho e Iván por ayudarnos en horas de trabajo.

A Alberto por esas tardes de risas y su colaboración en los videos.

A Oscar por su paciencia y su apoyo incondicional.

Y por último, pero no por ello menos importante a mi familia.

ÍNDICE

AGRADECIMIENTOS.....	1
ÍNDICE	3
ÍNDICE DE FIGURAS.....	4
RESUMEN	6
I. INTRODUCCIÓN	7
1.1. FILTRO DE KALMAN.	7
1.2. PROCESAMIENTO DE LA IMAGEN.	11
1.2.1 Detección de movimiento.....	12
1.2.2 Segmentación.....	16
1.2.3. Determinación de la posición.....	20
1.3. HERRAMIENTAS EMPLEADAS.....	21
1.3.1. Lenguaje de programación Matlab.	22
1.3.2 Visual C++.....	22
1.3.3. TRIPOD (Template for Real-Time Image PrOcessing Development).....	23
1.3.4. Logitech QuickCam SDK.....	24
1.4. ESTRUCTURA DEL DOCUMENTO.....	24
II. ANÁLISIS DE OBJETIVOS.....	25
III. TIEMPO REAL.....	27
IV. DESCRIPCIÓN INFORMÁTICA	29
4.1. ESPECIFICACIÓN DE REQUISITOS.	29
4.1.1. Requisitos Funcionales.....	29
4.1.2. Requisitos No Funcionales.	30
4.2. DISEÑO DEL PROYECTO.	31
4.3. DESCRIPCIÓN FUNCIONAL.	32
V. RESULTADOS EXPERIMENTALES.....	38
6.1. PRUEBAS Y RESULTADOS EN LA DETECCIÓN DE MOVIMIENTO.	38
6.2. PRUEBAS Y RESULTADOS DE SEGMENTACIÓN Y DETERMINACIÓN DE LA POSICIÓN.	40
6.3. PRUEBAS Y RESULTADOS EN EL SEGUIMIENTO DE OBJETOS.	42
6.3.1. Seguimiento de un objeto sobre fondo negro.....	42
6.3.2. Seguimiento de un objeto en fondo no controlado.....	45
6.3.3. Seguimiento de varios objetos sobre fondo negro.	47
6.3.4. Seguimiento de varios objetos en fondo no controlado.	48
6.3.5. Seguimiento de un objeto bajo condiciones de oclusión.....	50
6.3.6. Seguimiento de un objeto en tiempo real.....	53
VI. CONCLUSIONES Y AVANCES FUTUROS	55
VII. BIBLIOGRAFÍA	57

ÍNDICE DE FIGURAS

FIG.1 FILTRO DE KALMAN.....	11
FIG.2 EJEMPLO DE OBTENCIÓN DE LA IMAGEN DIFERENCIA.....	13
FIG.3 EJEMPLO DEL PROBLEMA DE LA APERTURA CONOCIDO COMO “OJO DE BUEY”.....	15
FIG.4: A) IMAGEN DE VARIOS OBJETOS CLAROS SOBRE FONDO OSCURO; B) HISTOGRAMA DE LA IMAGEN; C) IMAGEN UMBRALIZADA.	17
FIG.5 COMPARATIVA DE AMBOS TIPOS DE CONECTIVIDAD	18
FIG.6 PROYECCIÓN Y CENTRO DE MASAS DEL OBJETO.....	20
FIG.7: A) OBTENCIÓN DE LOS PUNTOS EXTREMOS DE UN OBJETO; B) OBTENCIÓN DEL CENTROIDE DEL OBJETO.....	21
FIG.8 INTERFAZ QUE PROPORCIONA TRIPOD, ESTE EJEMPLO MUESTRA LA BINARIZACIÓN DE LA IMAGEN CAPTURADA EN TIEMPO REAL.	23
FIG.9: (A) IMAGEN SINTÉTICA EN EL INSTANTE T; (B) IMAGEN SINTÉTICA EN EL INSTANTE T+1.	38
FIG.10 IMAGEN SINTÉTICA RESULTADO DE SUSTRAR A LA IMAGEN EN EL INSTANTE T+1, LA IMAGEN EN EL INSTANTE INMEDIATAMENTE ANTERIOR.....	38
FIG.11 IMAGEN RESULTADO DE LA SUMA DE LAS DIFERENCIAS DE DOS IMÁGENES CONSECUTIVAS DE UNA SECUENCIA DE VÍDEO, Y DEL CIERRE MORFOLÓGICO DE LA MISMA.....	39
FIG.12 EJEMPLOS DE SEGMENTACIÓN POR CONECTIVIDAD CON IMÁGENES SINTÉTICAS.	40
FIG.13: (A) SEGMENTACIÓN POR CONECTIVIDAD; (B) SEGMENTACIÓN POR CRECIMIENTO DE REGIONES.....	41
FIG.14 EJEMPLO DE DETECCIÓN DE MOVIMIENTO, SEGMENTACIÓN Y DETERMINACIÓN DE LA POSICIÓN DEL CENTRO DE MASAS DE UN OBJETO.....	41
FIG.15. EJEMPLO DEL SEGUIMIENTO DE UN OBJETO SOBRE UN FONDO NEGRO, CON VALORES $P_K=100$, $R_K=0.02$ Y $Q=10^{-3}$	42
FIG.16. EJEMPLO DEL SEGUIMIENTO DE UN OBJETO SOBRE UN FONDO NEGRO, CON VALORES $P_K=0.1$, $R_K=1$ Y $Q=10^{-3}$	43
FIG.17. EJEMPLO DEL SEGUIMIENTO DE UN OBJETO SOBRE UN FONDO NEGRO, CON VALORES $P_K=100$, $R_K=0.02$ Y $Q=10^{-3}$, MOSTRADO SOBRE LA IMAGEN DIFERENCIA.	44
FIG.18. EJEMPLO DEL SEGUIMIENTO DE UN OBJETO SOBRE UN FONDO NO CONTROLADO, CON VALORES $P_K=100$, $R_K=0.2$ Y $Q=10^{-3}$	45
FIG.19. EJEMPLO DEL SEGUIMIENTO DE UN OBJETO SOBRE UN FONDO NO CONTROLADO, CON VALORES $P_K=100$, $R_K=0.2$ Y $Q=10^{-3}$	46

FIG.20. SEGUIMIENTO DE DOS OBJETOS SOBRE FONDO NEGRO.	47
FIG.21 SEGUIMIENTO DE TRES PERSONAS EN FONDO NO CONTROLADO.....	48
FIG.22 SEGUIMIENTO DE DOS PERSONAS EN FONDO NO CONTROLADO.	49
FIG.23. SECUENCIA DE IMÁGENES CON OCLUSIÓN. $P_K=10$, $R_K= 0.2$ Y $Q=10^{-3}$	50
FIG.24. SECUENCIA DE IMÁGENES CON OCLUSIÓN. $P_K=0.1$, $R_K= 0.7$ Y $Q=10^{-3}$	51
FIG.25. SECUENCIA DE IMÁGENES CON OCLUSIÓN. $P_K=100$, $R_K= 0.02$ Y $Q=10^{-3}$	52
FIG.26 SEGUIMIENTO DE UNA PERSONA EN TIEMPO REAL.	53
FIG.27 SEGUIMIENTO DE UN OBJETO EN TIEMPO REAL.....	53
FIG.28 GRÁFICOS DE LA POSICIÓN OBSERVADA Y ESTIMADA PARA LOS VALORES $P_K=100$ Y $R_K=0.02$	54
FIG.29 GRÁFICOS DE LA DIFERENCIA EN LA POSICIÓN PARA LOS VALORES $P_K=100$ Y R_K $=0.02$	54
FIG.30 GRÁFICOS DE LA POSICIÓN OBSERVADA Y ESTIMADA PARA LOS VALORES $P_K=0.1$ Y $R_K=1$	54
FIG.31 GRÁFICOS DE LA DIFERENCIA EN LA POSICIÓN PARA LOS VALORES $P_K=0.1$ Y $R_K=1$	54

RESUMEN

La Visión Artificial es una disciplina que tiene como finalidad, la extracción automática de información del mundo físico a partir de imágenes. Dentro de la Visión Artificial tiene un significativo papel el seguimiento automático de objetos en movimiento.

El filtro de Kalman es una técnica recursiva para determinar los parámetros correctos de un sistema que evoluciona con el tiempo. Dados unos estimadores iniciales y los parámetros propios del sistema dinámico, el filtro va prediciendo y auto ajustándose con cada nueva medida.

El filtro de Kalman se utiliza en situaciones donde un proceso continuo es muestreado en intervalos de tiempo y tiene especial interés en problemas de seguimiento de objetos en secuencias de imágenes.

El objetivo principal de este proyecto recae en la aplicación del filtro de Kalman al seguimiento de objetos en secuencias de imágenes y en la evaluación de los resultados obtenidos en diversas condiciones experimentales.

Los resultados más importantes conseguidos han sido la aplicación del filtro de Kalman en secuencias de imágenes con más de un objeto y el control del seguimiento bajo situaciones de oclusión.

Por último, y a pesar de que no era un objetivo fundamental del proyecto, se ha conseguido implementar una aplicación del filtro de Kalman mediante técnicas de Visión Artificial, que es capaz de seguir un objeto en secuencias de imágenes en tiempo real.

I. INTRODUCCIÓN

La Visión Artificial es una disciplina que tiene como finalidad, la extracción de información del mundo físico a partir de imágenes, utilizando para ello un computador. Se trata de un objetivo ambicioso y complejo que actualmente se encuentra en una etapa primitiva[3].

Un sistema de Visión Artificial actúa sobre una representación de una realidad que le proporciona información sobre brillo, colores, formas, etcétera. Estas representaciones suelen estar en forma de imágenes estáticas, escenas tridimensionales o imágenes en movimiento[3].

La línea de investigación relacionada con el seguimiento de objetos móviles es uno de los temas centrales de la Visión Artificial. Partiendo de la suposición de que el movimiento de la escena observada es continuo, es posible hacer predicciones sobre el movimiento de los puntos de la imagen, en un instante determinado.

1.1. Filtro de Kalman.

Rudolf E. Kalman nació en Budapest en 1930, emigró a Estados Unidos durante la Segunda Guerra Mundial y se doctoró en el M.I.T. en Ingeniería Eléctrica en 1954. En 1960, R.E. Kalman publicó un famoso artículo describiendo una solución recursiva al problema de filtrado lineal de datos discretos. A partir de ahí, muchas investigaciones se han beneficiado de este trabajo, especialmente en campos como la navegación asistida o autónoma.

El filtro de Kalman proporciona un buen marco para la estimación de una variable, de la que se dispone de medidas a lo largo del tiempo. Se trata de una técnica de estimación Bayesiana empleada para seguir sistemas estocásticos dinámicos observados mediante sensores ruidosos.

En el ámbito de la Visión Artificial el filtro de Kalman es un algoritmo recursivo que se utiliza para estimar la posición de un punto o característica en movimiento y la incertidumbre de la medida, en la siguiente imagen. Se trata de buscar la característica (punto, borde, esquina, región, etc.) en un área determinada de la siguiente imagen alrededor de la posición predicha, en la que estamos seguros de encontrar la característica dentro de un cierto grado de confianza.

El objetivo del filtro es la obtención de un estimador óptimo de las variables de estado de un sistema dinámico, basado en observaciones ruidosas y en un modelo de incertidumbre de la dinámica del sistema.

El método estima el estado $\mathbf{x}$ perteneciente a $\mathcal{R}$ de un proceso controlado en tiempo discreto que es gobernado por la ecuación diferencial del tipo:

$$\mathbf{x}_{k+1} = \mathbf{A}_k \mathbf{x}_k + \mathbf{B} \mathbf{u}_k + \mathbf{w}_k \quad (1)$$

con una medida z correspondiente a la observación y perteneciente a $\mathcal{R}^m$ que es:

$$\mathbf{z}_k = \mathbf{H}_k \mathbf{x}_k + \mathbf{v}_k \quad (2)$$

Las variables aleatorias $\mathbf{w}_k$ y $\mathbf{v}_k$ representan el ruido del proceso y de la medida respectivamente y se asume que son independientes y blancos.

Cuando el filtro de Kalman se aplica a la Visión Artificial, el estado $\mathbf{x}$ se corresponde con el vector posición del objeto en la imagen determinado por las coordenadas de posición $\mathbf{x}_x$ y $\mathbf{x}_y$, y las coordenadas de velocidad $\mathbf{v}_x$ y $\mathbf{v}_y$. La observación $\mathbf{z}$ en cambio, es únicamente un vector de dos componentes $\mathbf{z}_x$ y $\mathbf{z}_y$, correspondiente a las coordenadas de la posición observada del objeto de interés.

La matriz $\mathbf{A}_{N \times M}$ relaciona el estado en tiempo k con el estado en tiempo $k+1$. Esta relación se manifiesta en las siguientes ecuaciones, dando como resultado la matriz $\mathbf{A}$.

$$\left\{ \begin{array}{l} x_{x_{k+1}} = x_{x_k} + v_x t \\ x_{y_{k+1}} = x_{y_k} + v_y t \\ v_{x_{k+1}} = v_{x_k} \\ v_{y_{k+1}} = v_{y_k} \end{array} \right. \quad A = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (3)$$

La matriz $\mathbf{B}_{N \times I}$ relaciona la entrada control $\mathbf{u}$ perteneciente a $\mathcal{H}$ con el estado $\mathbf{x}$.

Y la matriz $\mathbf{H}_{N \times M}$ relaciona el estado con la medida $\mathbf{z}_k$.

$$H_k = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

El filtro de Kalman proporciona una ecuación que computa un estimador del estado a posteriori $\hat{\mathbf{x}}_k$ como combinación lineal del estimador a priori $\hat{\mathbf{x}}_k^-$ y la diferencia ponderada entre la observación actual $\mathbf{z}_k$ y una predicción de medida $H_k \hat{\mathbf{x}}_k^-$:

$$\hat{\mathbf{x}}_k = \hat{\mathbf{x}}_k^- + K(\mathbf{z}_k - H_k \hat{\mathbf{x}}_k^-) \quad (4)$$

La diferencia $(\mathbf{z}_k - H_k \hat{\mathbf{x}}_k^-)$ se llama comúnmente *innovación de la medida* o simplemente *residuo* y refleja la discrepancia entre la predicción de la medida $H_k \hat{\mathbf{x}}_k^-$ y la observación actual $\mathbf{z}_k$. La matriz $\mathbf{K}_{N \times M}$ llamada *ganancia de Kalman* o factor de mezcla establece la cantidad de influencia del error entre nuestra estimación y la medida:

$$K_k = P_k^- H_k^T (H_k P_k^- H_k^T + R_k)^{-1} \quad (5)$$

Siendo P_k^- el estimador de la covarianza del error a priori y R_k la covarianza del error medido. Vemos que si R_k se aproxima a $\mathbf{0}$, la ganancia ponderará el residuo con mayor peso. Por el contrario, cuando P_k^- se aproxime a $\mathbf{0}$, la ganancia ponderará menos el residuo.

Otra forma más intuitiva de ver la ponderación de K , es que cuando la covarianza del error de medida R_k se aproxime a $\mathbf{0}$, tendremos más confianza en la observación actual z_k , mientras que la medida predicha $H_k \hat{x}_k^-$ perderá confianza en la misma medida. Por otra parte, cuando el estimador de la covarianza del error a priori P_k^- se aproxime a $\mathbf{0}$ se perderá confianza en la medida z_k y la de la medida predicha $H_k \hat{x}_k^-$ se incrementará.

En nuestro sistema P_k^- y R_k son matrices de dimensión 4x4, y Q_k es una matriz de 2x2. Matrices que inicializaremos con distintos valores para estudiar los diferentes resultados que proporcionan.

El filtro de Kalman estima variables de estado de un proceso con realimentación. Calcula el estado del proceso en algún instante y entonces obtiene información (se realimenta) de la medida. Por tanto, las ecuaciones del filtro se pueden clasificar en dos tipos: *actualización del tiempo* y *actualización de las medidas*. Las primeras son responsables de proyectar hacia el futuro los estimadores del estado actual y de la covarianza del error, para obtener los estimadores a priori del siguiente estado. Las ecuaciones de actualización de las medidas son responsables de la realimentación, incorporando una nueva medida a los estimadores a priori para obtener unos estimadores a posteriori mejorados. Las ecuaciones de actualización del tiempo pueden ser interpretadas como ecuaciones de *predicción*, mientras que las de actualización de la medida pueden pensarse como ecuaciones de *corrección*. Las ecuaciones específicas para las actualizaciones del tiempo y la medida son respectivamente:

$$\hat{x}_{k+1}^- = A_k \hat{x}_k + B u_k \quad (6)$$

$$\hat{P}_{k+1}^- = A_k P_k A_k^T + Q_k \quad (7)$$

y

$$K_k = P_k^- H_k^T (H_k P_k^- H_k^T + R_k)^{-1} \quad (8)$$

$$\hat{x}_k = \hat{x}_k^- + K (z_k - H_k \hat{x}_k^-) \quad (9)$$

$$P_k = (I - K H_k) P_k^- \quad (10)$$

La descripción del filtro de Kalman con sus ecuaciones puede verse en el siguiente diagrama.

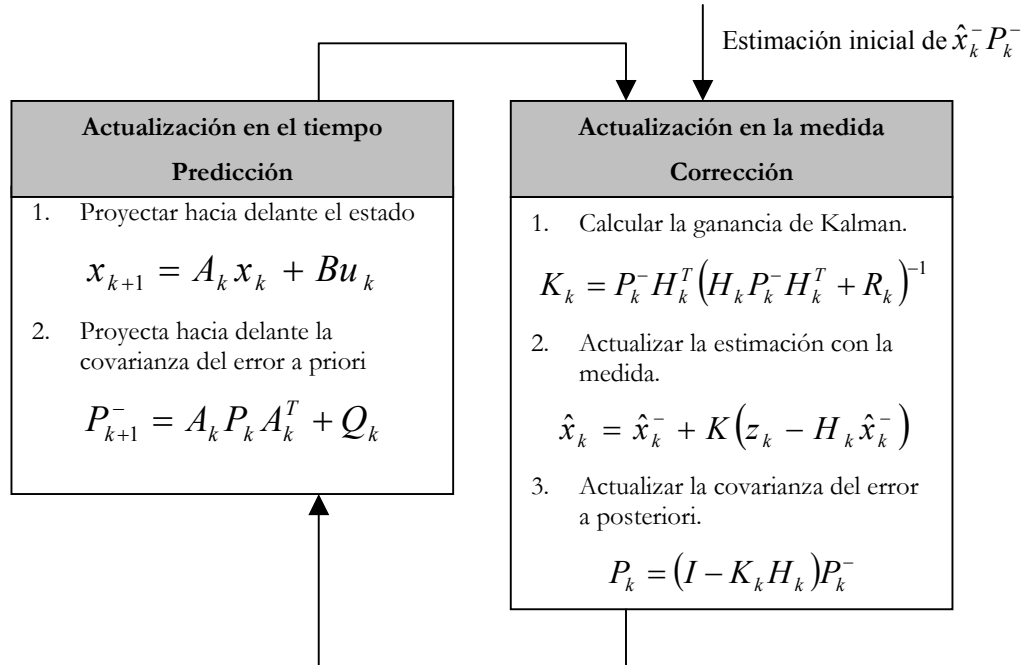


Fig.1 Filtro de Kalman.

1.2. Procesamiento de la imagen.

Para aplicar el filtro de Kalman al seguimiento de un objeto, es necesario proporcionar una característica representativa del mismo, que será tomada como la observación del objeto. Para el cálculo de este punto que denominaremos centro de masas y que determina la posición del objeto, es necesario realizar una serie de operaciones sobre la imagen. Abordaremos entonces los problemas de *detección de movimiento*, *segmentación*, y *determinación de la posición*.

1.2.1 Detección de movimiento.

Para identificar los objetos que han experimentado movimiento sobre el resto de la imagen (fondo), es necesario aplicar técnicas de detección de movimiento. De esta manera podremos seguir un móvil que se desplace sobre un fondo cualquiera.

Una imagen digital es una imagen $f(x,y)$ que puede ser discretizada en sus coordenadas espaciales y brillo. Por tanto, es una matriz cuyos índices identifican un punto en la imagen y la correspondiente matriz de valores identifica el nivel de gris de ese punto.

Para el estudio del movimiento en la imagen es imprescindible introducir la variable tiempo. Una secuencia de imágenes vendrá representada por la función $f(x,y,t)$, donde x e y son las coordenadas espaciales en la imagen de la escena en un instante de tiempo concreto t . El valor de la función $f(x,y,t)$, por tanto, representa la intensidad del píxel (x,y) en la imagen t [2].

El problema fundamental en el análisis de secuencias de imágenes está dado por el hecho de que un problema que por naturaleza es geométrico, tiene un efecto manifiesto en los cambios de intensidad en los píxeles que componen las imágenes; de tal forma, el problema de detección de movimiento se convierte en un problema de detección de cambios de intensidad en una secuencia de imágenes variables en el tiempo.

Uno de los métodos de detección de movimiento es el basado en el *cálculo de la imagen diferencia* f_d , que refleja los cambios ocurridos entre una imagen y la sucesiva en el tiempo. En caso de haberse producido movimiento, se obtiene entonces una imagen de bordes. Se define como:

$$f_d(p, t_1, t_2) = |f(p, t_2) - f(p, t_1)| \quad (11)$$

donde $p = (x,y)$ es un píxel de la imagen y t_1, t_2 son los instantes de tiempo de dos imágenes consecutivas.

Para entender mejor este método considérese el ejemplo de la Fig 2, donde se muestra una imagen en el instante t_1 que contiene una región cuadrada de intensidades f_p , moviéndose con velocidad $\mathbf{v}$, de módulo constante, dirección horizontal y sentido hacia la derecha, sobre un fondo homogéneo de distinta intensidad. La siguiente imagen en el instante t_2 contendrá esta región desplazada. La imagen diferencia contiene, por tanto, tres intensidades.

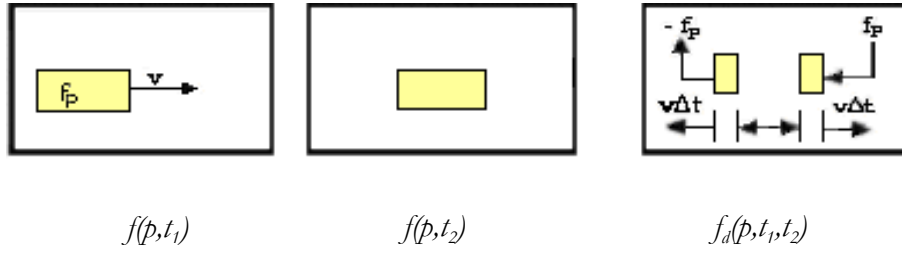


Fig.2 Ejemplo de obtención de la imagen diferencia.

Una variante de la Ec. (11) consiste en formar la imagen resultante f_{out} mediante:

$$f_{out} = \begin{cases} f(p, t_2) & \text{si } f_d(p, t_1, t_2) \geq T_d \\ 0 & \text{en otro caso} \end{cases} \quad (12)$$

donde T_d es un umbral de detección de cambio. A diferencia de la Ec. (11), esta operación permite retener sólo regiones de cambio significativo.

La mayor ventaja de esta técnica es su simplicidad. La información que proporciona, sin embargo, no es demasiado descriptiva respecto a la forma y movimiento de los objetos, aunque sí permite detectar la zona de la imagen donde se están produciendo cambios, pudiéndose así concentrar el posterior esfuerzo computacional en el área detectada.

En la aplicación en tiempo real hacemos una definición de la imagen diferencia que viene dada por las diferencias aparentes entre una primera imagen considerada fondo y el resto de las imágenes capturadas. Este método es equivalente a la sustracción de fondo.

Muchas de las técnicas más importantes para la estimación del movimiento están basadas en el cálculo del gradiente, esto es, en la observación del cambio de los niveles de intensidad en la imagen. El *flujo óptico* refleja los cambios de la imagen debido al movimiento durante un intervalo de tiempo, y el campo de flujo óptico es el campo de velocidad que representa el movimiento tridimensional de puntos de los objetos a través del movimiento bidimensional de la imagen [1].

La técnica del flujo óptico basada en el gradiente recurre a estimadores de las variaciones espaciales y temporales de las intensidades de los píxeles, para obtener un vector de velocidad instantánea en cada píxel y así conformar el campo de flujo óptico.

La hipótesis de partida es que las estructuras de intensidad de regiones en movimiento son aproximadamente constantes:

$$I(\vec{x}, t) = I(\vec{x} - \vec{v}t, 0) \quad (13)$$

donde $I(\vec{x}, t)$ es la intensidad luminosa del píxel p , cuyas coordenadas se definen en $\vec{x}$, en el tiempo t y $\vec{v} = (u, v)'$ es el vector de flujo óptico para dicho píxel p .

Matemáticamente se puede expandir en series de Taylor o bien asumir que

$$\frac{dI(\vec{x}, t)}{dt} = 0 \quad (14)$$

Aplicando entonces la regla de la cadena se llega a la Ecuación de Restricción del Flujo Óptico (E.R.F.O):

$$I_x u + I_y v + I_t = 0 \quad (15)$$

donde I_x , I_y e I_t son las derivadas parciales de I con respecto a x , y , y t , respectivamente. Los valores a determinar son u y v , y se tratan de resolver mediante la aplicación de la restricción de suavidad:

$$\left(\frac{\partial u}{\partial x}\right)^2 + \left(\frac{\partial u}{\partial y}\right)^2 + \left(\frac{\partial v}{\partial x}\right)^2 + \left(\frac{\partial v}{\partial y}\right)^2 = 0 \quad (16)$$

Finalmente para obtener la velocidad de la imagen se utilizan métodos numéricos iterativos:

$$u^{n+1} = \bar{u}^n - \frac{I_x(I_x \bar{u}^n + I_y \bar{v}^n + I_t)}{\lambda + I_x^2 + I_y^2} \quad (17)$$

$$v^{n+1} = \bar{v}^n - \frac{I_y(I_x \bar{u}^n + I_y \bar{v}^n + I_t)}{\lambda + I_x^2 + I_y^2} \quad (18)$$

donde n denota el número de iteración, la velocidad (u^0, v^0) se inicializa a cero, y $\bar{u}^n$ y $\bar{v}^n$ son medidas de los vecinos de u^n y v^n . I_x e I_y son las imágenes gradiente para los ejes x e y respectivamente, I_t es la imagen diferencia y λ es un parámetro experimental.

Esta solución iterativa mitiga en cierta medida el *problema de apertura*. Para entender este problema consideremos un contorno en una imagen y pensemos que sólo tenemos una ventana de visibilidad alrededor de un punto de interés (x, y) en el contorno. Si la posición del contorno cambia debido al movimiento respecto a la cámara, entonces no se puede decir con total seguridad en qué dirección se ha movido, basándonos únicamente en la información local disponible en la ventana.

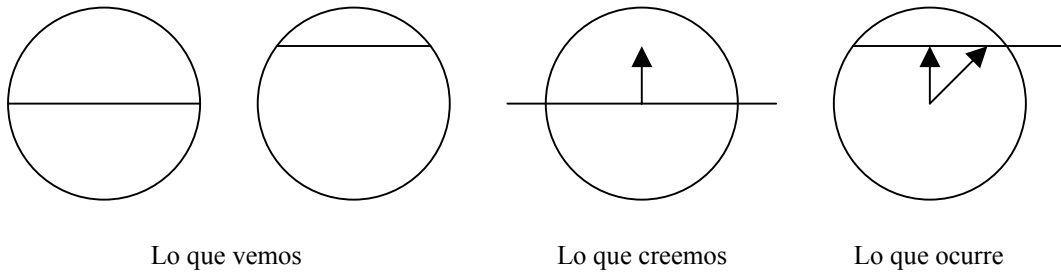


Fig.3 Ejemplo del problema de la apertura conocido como “ojo de buey”

1.2.2. Segmentación.

La segmentación es el proceso que divide una imagen en regiones u objetos cuyos píxeles poseen atributos similares, y constituye uno de los procesos más importantes de un sistema automatizado de visión, ya que permite extraer los objetos de la imagen para su posterior descripción y reconocimiento [2].

La segmentación es necesaria en nuestra aplicación porque nos permite crear una región de interés por cada movimiento acaecido. De esta manera podemos realizar el seguimiento para varios objetos, procesando cada región de interés de manera independiente.

Las distintas técnicas de segmentación pueden encuadrarse en tres grupos fundamentales: técnicas de umbralización, técnicas basadas en regiones y técnicas basadas en la detección de bordes.

El método de *segmentación por umbralización* tiene en cuenta el valor de intensidad de los píxeles, para decidir si estos forman parte de un objeto de interés. En una imagen en escala de grises se debe encontrar el rango de valores de intensidad que caracterizan dicho objeto, lo que requiere entonces el análisis del histograma de la imagen.

El objetivo de este método, es encontrar de una manera óptima los valores característicos de la imagen que establecen la separación del objeto de interés, con respecto a las regiones que no pertenecen al mismo. Si los valores de intensidad del objeto y del resto de la imagen difieren claramente, entonces el histograma mostrará una distribución bimodal, con dos máximos distintos, separados por una región vacía, con lo cual se logrará una separación perfecta entre el objeto y el fondo, al establecer un valor umbral ubicado en esta región del histograma. Por lo tanto cada píxel de la imagen, es asignado a una de dos categorías, dependiendo si el valor umbral es excedido o no.

Considérese el histograma de la Figura 4b correspondiente a la imagen de la figura 4a, compuesta por varios objetos claros sobre fondo oscuro. Una forma de extraer el objeto de la imagen es seleccionar un umbral de intensidad T por encima del cual se encuentra todos los píxeles pertenecientes al objeto. De este modo, cualquier punto (x,y) , para el que $f(x,y) > T$ es un punto del objeto, mientras que si $f(x,y) \leq T$ será un punto del fondo.

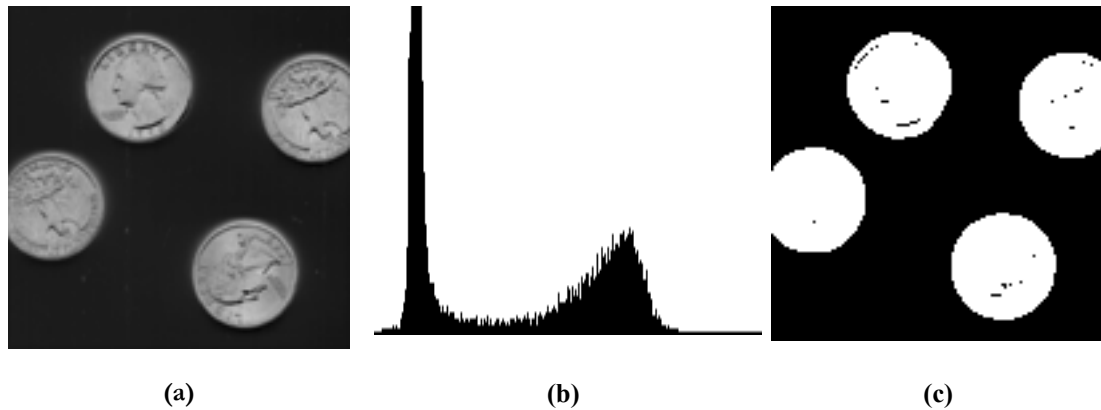


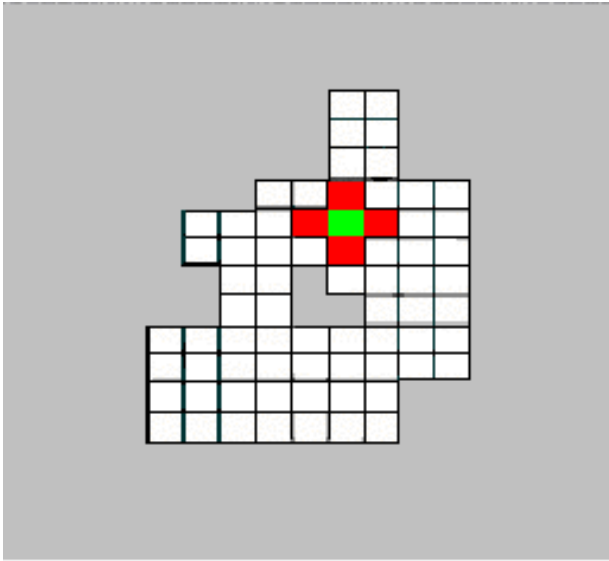
Fig.4: a) Imagen de varios objetos claros sobre fondo oscuro; b) Histograma de la imagen; c) Imagen umbralizada.

Los *métodos de segmentación basados en regiones* tienen en cuenta un conjunto de píxeles de la imagen, en los que se analizan características como la posición en el espacio de intensidades, relaciones topológicas (conectividad) y características de las fronteras entre dos conjuntos. Dependiendo de como sea analizada la posición en el espacio y las relaciones espaciales existentes entre los píxeles, se pueden encontrar *métodos de Clasificación* y métodos por *Crecimiento de Regiones*.

Los *métodos de Clasificación* determinan primero una partición del espacio de intensidades y utilizan luego las relaciones de conectividad para determinar una región.

La *segmentación basada en conectividad de regiones* considera que una región en una imagen está constituida por un conjunto de píxeles contiguos y conectados. La conectividad de un píxel con otro puede ser de 4-conectividad si existe relación entre dos píxeles colindantes horizontalmente o verticalmente, o bien 8-conectividad si los píxeles se tocan en forma diagonal también. Dos píxeles que no son colindantes también pueden tener 4 u 8-conectividad si existe un camino de uno al otro a través de píxeles conectados.

4-CONECTIVIDAD



8-CONECTIVIDAD

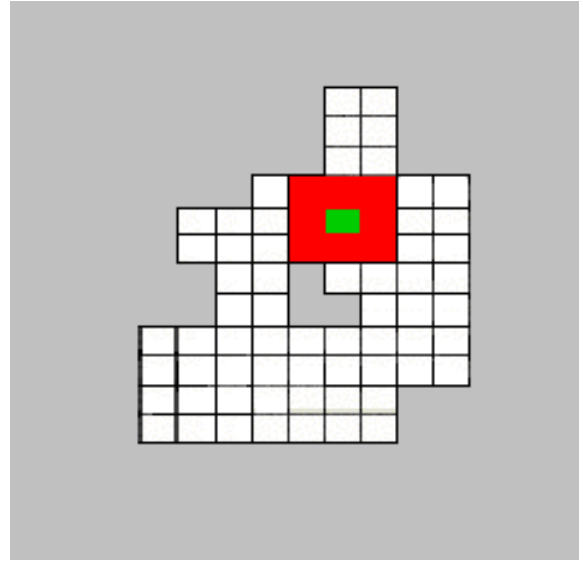


Fig.5 Comparativa de ambos tipos de conectividad

El mapeo de objetos se puede realizar evaluando los niveles de intensidad sobre una imagen en escala de grises o bien evaluando negros y blancos sobre una imagen binaria.

En el procesamiento de la imagen determinamos, dependiendo de un umbral, si cada píxel es miembro o no de un objeto etiquetándolo. Un píxel a considerar puede ser el primer píxel de un objeto nuevo, o puede pertenecer a un objeto ya identificado, en cuyo caso presenta conectividad con píxeles de ese objeto.

Una vez identificados todos los píxeles de cada objeto, determinaremos la situación de estos objetos en la imagen, con la posición del primer y último píxel perteneciente a cada objeto, que determinarán la región de interés a recorrer.

Los *métodos de Crecimiento de Regiones* utilizan de manera simultánea tanto la partición del espacio de intensidades como el estudio de la relación de conectividad, para determinar una región.

Este método, considera a cada píxel como un nodo de un grafo, luego analizando una vecindad de un píxel cualquiera, se unen por medio de un arco, todos aquellos píxeles con características similares. La intensidad de los niveles de gris es la característica de similitud buscada, de forma que dos píxeles estarán conectados si estos difieren en menos de un umbral preestablecido, y si se encuentran en una determinada vecindad.

La segmentación de una imagen puede también llevarse a cabo mediante la *detección de los límites de cada región (detección de bordes)*, es decir, localizando los lugares donde se produce un cambio significativo de los niveles de intensidad de los píxeles (detección de bordes). Esta operación puede efectuarse utilizando operadores de detección de bordes basados en la primera y/o segunda derivada. En el primer caso, como consecuencia del ruido, iluminación no uniforme y otros, los operadores gradiente rara vez definen por completo la frontera de los objetos, necesitándose, en tal caso, algoritmos que se encarguen de realizar la unión de los píxeles detectados mediante el seguimiento del contorno [2].

1.2.3. Determinación de la posición.

Para seguir un objeto es necesaria una medida de observación que corresponderá con la posición del centro de masas del objeto. Para ello proponemos dos métodos, uno basado en proyecciones y otro en los extremos del objeto.

La *determinación de la posición basada en proyecciones* se sirve del análisis del histograma de la imagen para ambos ejes de coordenadas x e y . Definimos como el centro de masas a aquel punto que viene dado en su coordenada x por el punto medio de la proyección en el eje de abscisas, y en su coordenada y por el punto medio de la proyección en el eje de ordenadas.

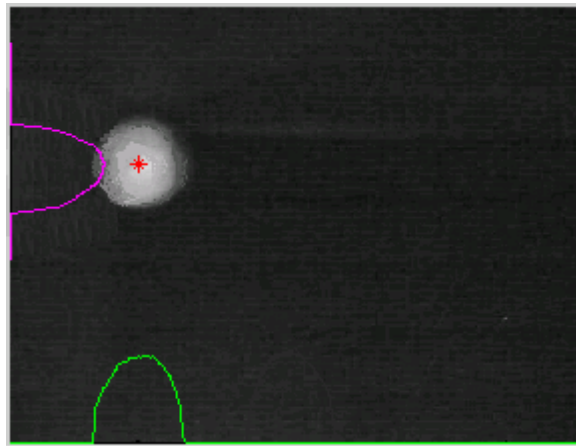


Fig.6 Proyección y centro de masas del objeto.

Previamente se elimina posible ruido, teniendo en cuenta solamente a aquellos puntos de la imagen con más de un número fijo de píxeles superiores al umbral establecido. De esta manera nos aseguramos de no considerar como objeto algún píxel aislado.

La *determinación de la posición basada en los extremos de un objeto* se fundamenta en la obtención de los vértices, izquierdo (x_1, y_1) , derecho (x_2, y_2) , superior (x_3, y_3) e inferior (x_4, y_4) del objeto de la imagen (Fig.7a).

La localización del objeto vendrá determinada por un punto único característico del mismo (x_{obj}, y_{obj}) , que obtendremos como resultado de la intersección de las diagonales de caja contenedora (Fig.7b).

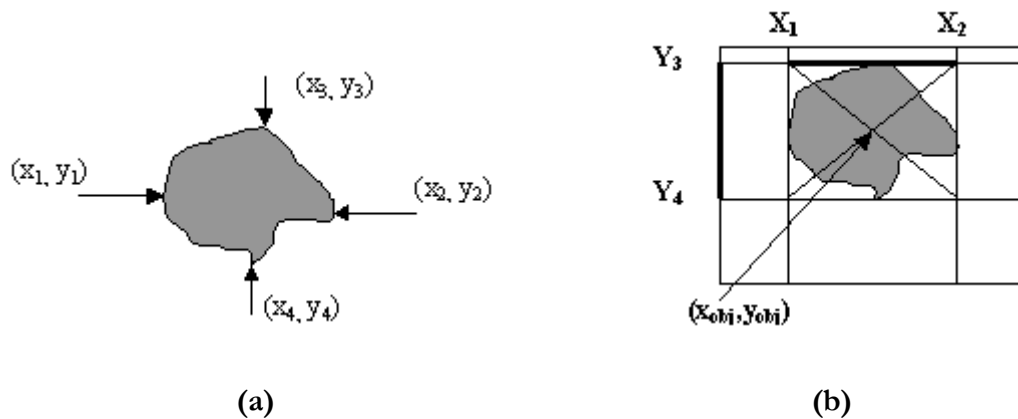


Fig.7: a) Obtención de los puntos extremos de un objeto; b) Obtención del centroide del objeto.

1.3. Herramientas empleadas.

Las herramientas software utilizadas para la realización de este proyecto son:

- Matlab 6.1 y su Toolbox de procesamiento de imágenes.
- Visual C++.
- Proyecto TRIPOD para el desarrollo de la aplicación en tiempo real.
- Logitech QuickCam SDK.

1.3.1. Lenguaje de programación Matlab.

MATLAB es el nombre abreviado de “MATrix LABoratory”. Se trata de un programa para realizar cálculos numéricos con **vectores** y **matrices**. Como caso particular puede también trabajar con números escalares, tanto reales como complejos. Una de las capacidades más atractivas es la de realizar una amplia variedad de **gráficos** en dos y tres dimensiones. Tiene también un lenguaje de programación propio. Es un gran programa para cálculo técnico y científico. Para ciertas operaciones es muy rápido, cuando puede ejecutar sus funciones en código nativo. En otras aplicaciones resulta bastante más lento que el código equivalente desarrollado en C/C++ o Fortran. Sin embargo, siempre es una magnífica herramienta de alto nivel para desarrollar aplicaciones técnicas, fácil de utilizar y que aumenta la productividad de los programadores respecto a otros entornos de desarrollo.

MATLAB dispone de un código básico y de varias librerías especializadas denominadas **Toolboxes**. Estas Toolboxes cubren las áreas principales en el mundo de la ingeniería y la simulación, destacando entre ellas la de proceso de imágenes, que ha sido imprescindible para este proyecto.

1.3.2 Visual C++.

Visual C++ es un entorno de desarrollo de software, destinado específicamente para escribir programas para el sistema operativo Windows en C++. Con Visual C++, se puede escribir cualquier tipo de programa, que es posible implementar tanto en lenguaje C++, como en C. C++ es un superconjunto de C orientado a objetos. Visual C++ abarca una gran cantidad de temas: lenguaje C++, programación orientada a objetos (POO), biblioteca de clases, tecnología de componentes, etc.

Uno de los propósitos de Visual C++ es programar para Windows en C++ utilizando MFC. MFC es un conjunto de clases de C++ que comprenden una aplicación genérica de funcionamiento de Windows lista para personalizar.

1.3.3. TRIPOD (Template for Real-Time Image PrOcessing Development).

Aplicación para el Desarrollo del Procesamiento de Imágenes en Tiempo Real, que su creador Paul Y. Oh, denomina TRIPOD. Se trata de un software de libre distribución. Fue desarrollado para permitir a los programadores procesar rápidamente imágenes en tiempo real capturadas con una webcam Logitech bajo entorno Windows, ya que ésta proporciona el software QuickCam SDK. TRIPOD está formado por archivos fuente del MFC de Visual C++ por encima de los cuales se integra el programa de procesamiento de imágenes para obtener resultados.

La interfaz que muestra el sistema está constituida por una ventana superior que exhibe las imágenes en color tomadas con una webcam Logitech, mientras que la ventana inferior muestra el resultado del procesamiento realizado sobre cada imagen capturada.

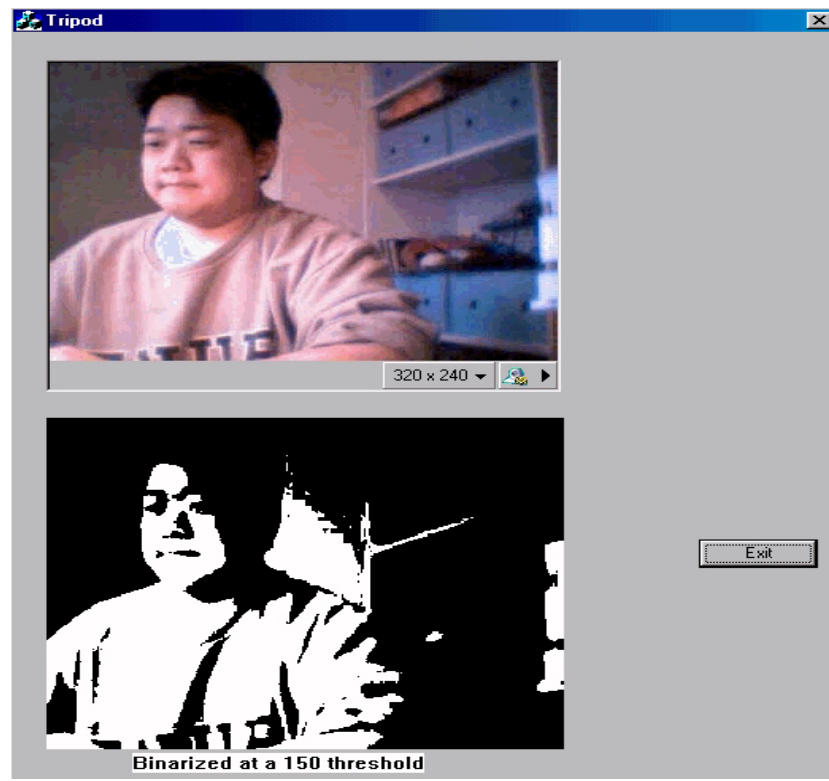


Fig.8 Interfaz que proporciona Tripod, este ejemplo muestra la binarización de la imagen capturada en tiempo real.

1.3.4. Logitech QuickCam SDK.

El motivo del uso de una cámara web Logitech es para poder apreciar las ventajas del kit de desarrollo software QuickCam (QCSDK) que ésta proporciona.

SDK también denominado API es una interfaz para programas de aplicación que proporciona una serie de reglamentos y acuerdos que definen la manera de llamar determinado servicio desde cierto programa.

Con QCSDK se puede desarrollar fácil y rápidamente aplicaciones como la realización de fotos y archivos de vídeo con extensión AVI. Además permite cualquier número de conexiones simultáneas de la cámara fotográfica a través de la interfaz. Las imágenes que proporciona QCSDK son RGB de color verdadero (24 bits) .

1.4. Estructura del documento.

El resto del documento se compone de 5 partes más, el siguiente apartado refleja los objetivos del proyecto. Un tercer apartado detalla la aplicación en tiempo real. Un cuarto capítulo recoge la descripción informática. Una quinta parte muestra las pruebas y resultados obtenidos. Y por último en la sexta parte se explican las conclusiones y se reflexiona sobre futuros avances.

II. ANÁLISIS DE OBJETIVOS

El objetivo principal del proyecto consiste en *implementar el Filtro de Kalman para seguimiento en secuencias de imágenes 2D, evaluando los resultados que se obtienen en la aplicación bajo diferentes condiciones experimentales*, como oclusión, movimientos no plano paralelos, y varios objetos. La consecución de este propósito se puede desglosar en los siguientes objetivos operativos:

- Familiarizarse con la herramienta de programación Matlab 6.1., y con el tratamiento de imágenes.
- Implementar funciones que abordan la detección de movimiento, la segmentación, etc., con imágenes sintéticas.
- Coleccionar diferentes secuencias de imágenes para evaluar el filtro de Kalman bajo distintas condiciones experimentales.
- Proyectar los conocimientos adquiridos en el estudio con imágenes sintéticas al tratamiento de imágenes reales.
- Detectar movimiento, segmentar la imagen, y localizar la posición del centroide del objeto en secuencias de imágenes reales.
- Estudiar la respuesta del filtro ante situaciones de oclusión o movimientos no plano paralelos.
- Seguir varios objetos móviles dentro de una misma escena.

Un objetivo añadido a posteriori, que no formaba parte de la idea original del proyecto, fue *implementar la aplicación en tiempo real*, propuesta que planteó la necesidad de:

- Comprender el funcionamiento de la plantilla TRIPOD.
- Familiarizarse con el entorno de programación Visual C++, y los lenguajes de programación C/C++.
- Contemplar ciertas restricciones como la realización de la sustracción de fondo, en lugar de la detección de movimiento.

III. TIEMPO REAL

“Un sistema de tiempo real es aquel en el que para que las operaciones computacionales estén correctas no depende sólo de que la lógica e implementación de los programas computacionales sean correctos, sino también en el tiempo en el que dicha operación entregó su resultado. Si las restricciones de tiempo no son respetadas el sistema se dice que ha fallado[4].”

Cualquier sistema desarrollado en tiempo real tiene que responder a estímulos generados externamente dentro de un plazo especificado y finito. Su funcionamiento correcto no sólo depende de los resultados del cálculo, sino también del instante en el que se generan estos resultados. Más que ser rápido, un sistema a tiempo real debe ser *predecible*.

El estudio del filtro de Kalman para seguimiento en tiempo real no era objetivo inicial del desarrollo del proyecto, pero en sistemas de control, medición, comunicaciones, etc., es tan importante llevar a cabo la tarea para la cual fueron diseñados como cumplirla en un tiempo preciso.

Para la realización del proceso de seguimiento en tiempo real se ha hecho uso de una implementación capturadora de imágenes, denominada TRIPOD (**(T)**emplate for **(R)**eal-Time **(I)**mage **(P)**r**(O)**cessing **(D)**evelopment), que proporciona al programador la posibilidad de procesar imágenes en tiempo real. Para utilizar TRIPOD fue necesario desarrollar la implementación del proceso en Visual C++, valiéndonos de los lenguajes de programación C y C++.

Nuestra aplicación tiene como objetivo realizar el seguimiento de un objeto que se mueve al largo de una escena. En cada imagen mostramos la observación del objeto, caracterizada por el **centro de masas** del mismo y representada por un punto rojo, la **estimación** que se obtiene como resultado del filtro de Kalman, que caracteriza la posición predicha y se ilustra como un punto de color verde, y la **región de interés** que se calcula a partir de la estimación y del error de la medida, y que mostramos mediante un cuadrado azul celeste.

Las diferencias entre el proceso desarrollado en tiempo real y el realizado con videos (formato .avi) son:

1. En la aplicación desarrollada en tiempo real no realizamos detección de movimiento, sino sustracción de fondo. De esta manera, aunque el objeto se quede parado en la escena, la observación, el estimador y la región de interés seguirán encuadrando al objeto, y no se considerará que hay oclusión como ocurre en el procesamiento mediante videos. Es un sistema que no reacciona ante cambios de movimiento sino ante cambios con respecto a una primera imagen fondo, lo que supone una restricción en el proceso.
2. En el proyecto implementado en tiempo real hemos realizado segmentación mediante umbralización para un único objeto.
3. En la aplicación desarrollada en Matlab era necesario que cada cierto tiempo hiciéramos el procesamiento para toda la imagen, de esta manera éramos capaces de detectar la aparición de nuevos elementos en la escena. En cambio en el proyecto a tiempo real planteamos el seguimiento para un único objeto, y aunque se pueda pensar que procesar toda la imagen cada cierto tiempo nos puede ayudar en caso de oclusión, o en caso de que el objeto se mueva muy deprisa saliéndose de la región de interés, esta medida no es necesaria dado que en estas situaciones la ventana de interés crece hasta volver a encontrar al objeto.

El resto del proceso es idéntico para ambos métodos.

IV. DESCRIPCIÓN INFORMÁTICA

4.1. Especificación de requisitos.

4.1.1. Requisitos Funcionales.

Entendemos cómo tales aquellos requisitos que detallan **qué** debe realizar el sistema. En nuestra aplicación consideramos los siguientes:

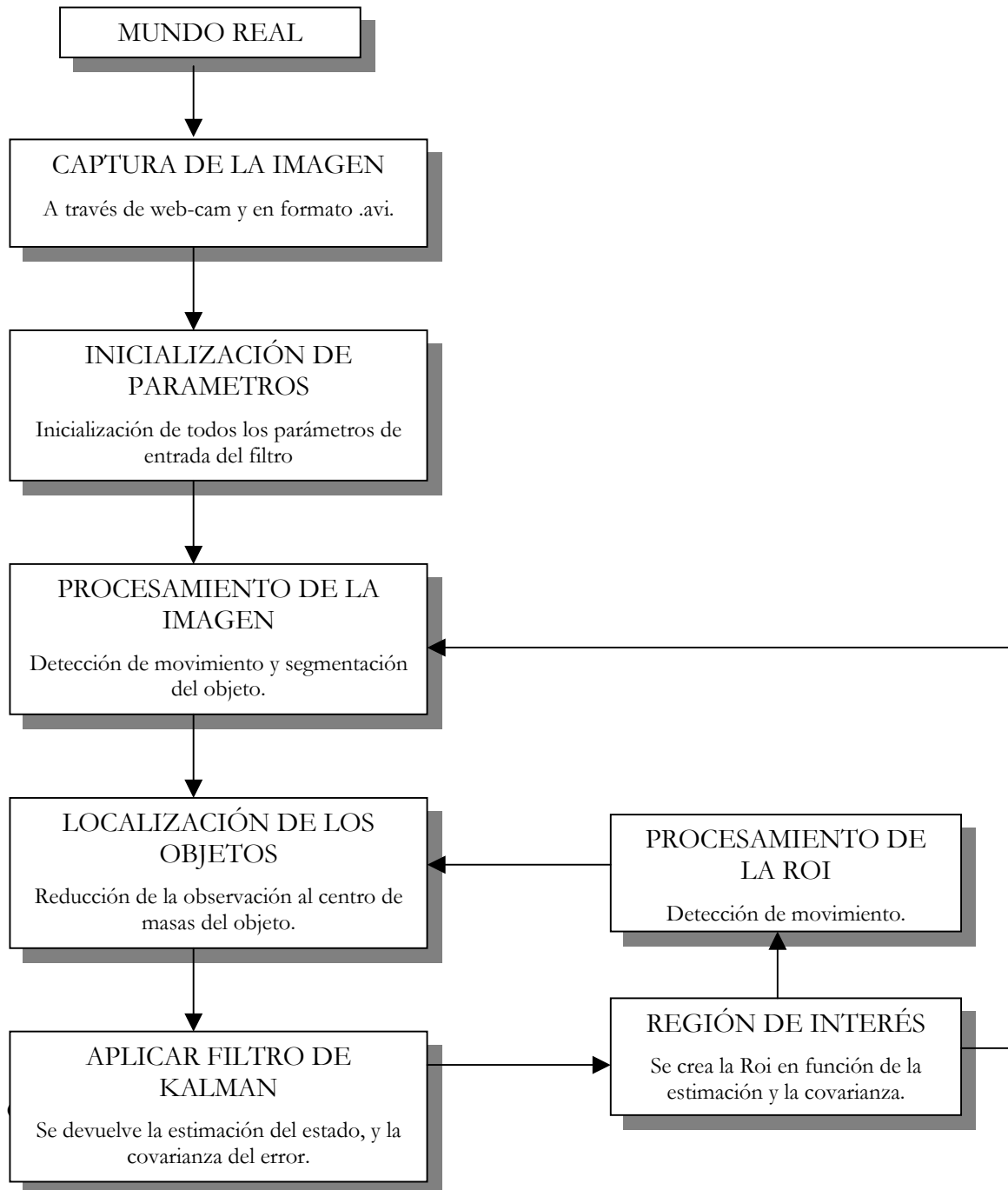
- En la realización del proyecto en tiempo real el sistema debe captar imágenes y procesarlas de manera inmediata.
- La detección de movimiento debe extraer las diferencias entre una imagen y su consecutiva.
- La segmentación debe determinar las dimensiones del objeto móvil.
- La localización del objeto debe determinar la posición del centro de masas del mismo.
- El filtro de Kalman debe devolver una estimación de la posición futura del objeto lo más próxima posible a su centro de masas, así como las dimensiones de una ventana de interés que lo encuadre.
- No se deben dar estados donde la región de interés no encuadre al objeto, a no ser que se trate de un caso de oclusión, que la velocidad del objeto sea muy elevada.

4.1.2. Requisitos No Funcionales.

Aquí indicamos los requisitos que detallan **cómo** debe responder el sistema y lo que se espera de él.

- Debe reaccionar rápidamente al movimiento del objeto.
- La visualización de los resultados debe ser intuitiva, es decir, dar la seguridad de que se está realizando el seguimiento de objetos.
- Una restricción del sistema está relacionada con la posición relativa de los objetos en la imagen, ya que si éstos se encuentran muy próximos el sistema no es capaz de diferenciar unos de otros.
- En la aplicación implementada en Visual Studio 6.0. para el desarrollo del proyecto en tiempo real se realizó la detección del objeto empleando el método sustracción de fondo, lo que supuso otra restricción en el proceso ya que necesitamos la captura de una primera imagen que tomaremos como fondo de la escena.

4.2. Diseño del proyecto.



4.3. Descripción funcional.

Para el correcto funcionamiento de todo el proceso (detección, segmentación, localización, seguimiento, y visualización) en el sistema implementado en Matlab 6.1., precisamos del uso de distintas funciones que describimos a continuación.

Proyecto.m

Objetivo: Realizar y mostrar el seguimiento de los objetos en movimiento.

Entradas: La secuencia de imágenes que se va a analizar y las constantes que multiplicarán a R , Q y P_{k1} (errores).

Salidas: Muestra la secuencia de imágenes resultado del seguimiento, y existe la posibilidad de recogerla en un vídeo formato AVI.

Descripción:

- Carga la película, a la cual se va aplicar el proceso.
- Inicializa los parámetros de entrada al filtro (X_k , P_{k1} , H_k , A_k , R , Q) haciendo uso de la función `IniciarVariables.m`, que además proporciona el umbral deseado.
- Si se trata de las primeras imágenes de la secuencia, y para cada cierto número de veces:
 - Llama a la función `DeteccionDeMovimiento.m` para obtener la imagen diferencia de toda la imagen.
 - Segmenta la imagen para calcular la región de interés, devolviendo así, el vector de coordenadas de los objetos, y el número de objetos que hay en la escena. El cálculo se puede resolver con la función `SegmentacionConexas.m` o con la función `SegmentacionCrecRegion.m`, ya que fueron implementados ambos tipos de segmentación.
- Para todos los frames de la secuencia, y para todas las ventanas de interés que encuadran a cada objeto de la imagen:
 - Detecta movimiento dentro de la región de interés (llama a la función `DeteccionDeMovimiento.m`).
 - Llama a la función `CentroDeMasas.m` que determina el centro de masas de cada objeto.
 - Obtiene la actualización de la estimación (X_k) y la covarianza del error (P_{k1}) que proporciona `FiltroKalman.m`
 - Crea una nueva región de interés donde realizar el procesamiento de la imagen siguiente apoyándose en la función `Roi.m`.

IniciarVariables.m

Objetivo: Inicializar las variables de entrada al filtro, y solicitar el principio y final de la secuencia en la que se desea aplicar el filtrado, así como el umbral que determina la frontera entre lo que consideramos objeto y lo que tomamos como fondo.

Entradas: La secuencia de imágenes que se va a analizar y las constantes que multiplicarán a R , Q y P_{k1} .

Salidas: El principio y final de la secuencia, el umbral, R , Q , P_{k1} , H_k , A_k , y X_k .

Descripción:

- Inicializa los valores de P_{k1} (matriz de 4×4), R (matriz de 2×2), Q (matriz de 4×4), H_k (matriz de 2×4), y A_k (matriz de 4×4).
- Solicita el umbral por pantalla.
- Calcula la velocidad inicial e inicializa el valor de la estimación $X_k=(\text{posicion_x}, \text{posicion_y}, \text{velocidad_x}, \text{velocidad_y})$.

DeteccionDeMovimiento.m

Objetivo: Obtener el movimiento acaecido en una secuencia de imágenes, esto es, la diferencia de posición de los objetos entre un frame y su predecesor.

Entradas: La secuencia de imágenes que se va a analizar, el frame para el se quiere detectar movimiento, y las coordenadas de la región donde detectar movimiento.

Salidas: La imagen diferencia.

Descripción:

- Convierte a escala de grises la imagen actual y su predecesora.
- Resta una imagen y consecutiva, así como esta con su predecesora.
- Suma las imágenes diferencia.
- Aplica el cierre morfológico a la imagen suma.

FiltroKalman.m

Objetivo: Dados unos estimadores iniciales, predecir la estimación y la covarianza del error, auto ajustándose con cada nueva medida.

Entradas: X_k , Z_k , H_k , P_{k1} , A_k , Q , R .

Salidas: La estimación de la posición X_k , y la covarianza del error P_{k1} .

Descripción:

- Calculamos la proyección de la covarianza del error, P_k (predicción).
- Calculamos la ganancia K_k para los nuevos valores de P_k .
- Actualizamos el estimador X_k , con la medida Z_k .
- Actualizamos la covarianza del error, P_{k1} (corrección).

SegmentacionConexas.m

Objetivo: Proporcionar las coordenadas de cada objeto no conectado.

Entradas: La imagen diferencia.

Salidas: Las coordenadas de la región de interés y el número de objetos no conectados.

Descripción:

- Binariza la imagen con factor 0.2.
- Etiqueta cada objeto de la matriz, y obtenemos el número de objetos de la imagen.
Consideramos conectividad-8.
- Para todos los objetos de la imagen:
 - Localiza sus coordenadas.

SegmentcionCrecRegion.m

Objetivo: Proporcionar las coordenadas de cada ventana de interés donde se localizan los objetos. El tamaño de la ventana evoluciona progresivamente desde el primer píxel que corresponde a un objeto hasta el último.

Entradas: La imagen diferencia.

Salidas: Las coordenadas de la región de interés y el número de objetos de la imagen.

Descripción:

- Recorre la imagen evaluando el valor de los píxeles.
- Cuando un píxel supere el umbral establecido, se toma su posición y se evalúa el valor de los píxeles próximos, si estos siguen rebasando el umbral se evalúan los próximos a estos y así progresivamente se aumenta el tamaño de la ventana, hasta que ninguno de estos píxeles vecinos supere el umbral.

Roi.m

Objetivo: Diseñar la ventana (región de interés) en función de la estimación y la covarianza del error proporcionadas por el filtro de Kalman.

Entradas: La imagen actual, el vector de estimaciones X_kV , los vectores de coordenadas de la observación ZxV , y ZyV , y una copia local de la covarianza del error P_{k1} .

Salidas: Las coordenadas de la nueva región de interés.

Descripción:

- Se calcula el máximo entre el factor heurístico de tolerancia (una constante), y la raíz cuadrada de la componente (1,1) de la matriz de covarianza P_{k1} , y entre la componente (2,2).
- Para cada estimación obtenida del filtro:
 - Se obtiene una región de interés alrededor de la estimación, sumando y restando el máximo calculado, a cada componente x e y de la estimación (X_k).
 - Controla el tamaño de la ventana obtenida para que no supere los márgenes de la imagen.
- Llama a la función que muestra el resultado del proceso MostrarRes.m.

CentroDeMasas.m

Objetivo: Obtener el centro de masas del objeto en la observación, recorriendo la región de interés. Si en un momento determinado se produce oclusión, esto es, que desaparece el objeto, devolver como observación la última estimación proporcionada por el filtro, y aumentar la ventana de búsqueda, creciendo la covarianza del error de manera local (sin afectar al filtro).

Entradas: La imagen actual, las coordenadas de la región donde localizar la posición del objeto, la estimación anterior, la covarianza del error P_{k1} , una copia local de la covarianza del error y el umbral que determina donde comienzan y terminan las dimensiones del objeto.

Salidas: La posición del centro de masas del objeto, una copia local de la covarianza del error P_{k1} modificada en caso de oclusión.

Descripción:

- Evalúa cada ventana de interés de la imagen almacenando en los vectores V_x y Y_x , el número de píxeles de cada columna y de cada fila, respectivamente, que superen el umbral.
- Recorre cada vector, parando en la primera ocasión que localiza un valor que se establezca suficiente como medida para eliminar ruido. Se toma entonces la posición de este primer valor. También toma la posición del último valor que lo cumpla.
- Resta la última posición la primera, obtenemos así, el **alto** y **ancho** del objeto.
- Divide el resultado entre 2.
- Suma por último el valor obtenido de la división, con la posición del primer valor considerado. Este punto, será el centro de masas del objeto.
- Si no hay objeto se toma la posición del centro de masas como la estimación anterior, y se aumenta el valor de la covarianza de manera local, para así, aumentar el tamaño de la ventana de búsqueda.

En el proyecto aplicado en *tiempo real* fue necesario la implementación de distintas funciones para el manejo de matrices, ya que el filtro de Kalman necesita de ciertas operaciones matriciales como la suma, resta, multiplicación y cálculo de la inversa para su correcto desarrollo.

Como ya hemos mencionado con anterioridad para la aplicación desarrollada en tiempo real empleamos la técnica de *sustracción de fondo*. Este método de detección de objetos, se efectuó evaluando las diferencias entre una primera imagen capturada, considerada fondo, y la imagen actual donde se pretende detectar el objeto. De esta manera creamos una imagen diferencia cuyos valores de los píxeles vendrán determinados en función de las diferencias entre los píxeles de ambas imágenes. Es decir, si un píxel de la imagen fondo dista en más de un determinado valor del píxel correspondiente en la imagen actual, se le otorgará un valor correspondiente a blanco en la imagen diferencia, o bien correspondiente a negro en caso de no superar el umbral.

Para el resto de la implementación podemos afirmar que la descripción funcional es equiparable a la especificada en Matlab.

V. RESULTADOS EXPERIMENTALES

6.1. Pruebas y resultados en la detección de movimiento.

Para familiarizarnos con el tratamiento de imágenes y en este caso con la detección de movimiento, se realizaron estudios previos con secuencias de imágenes sintéticas. (Fig9 y Fig.10).

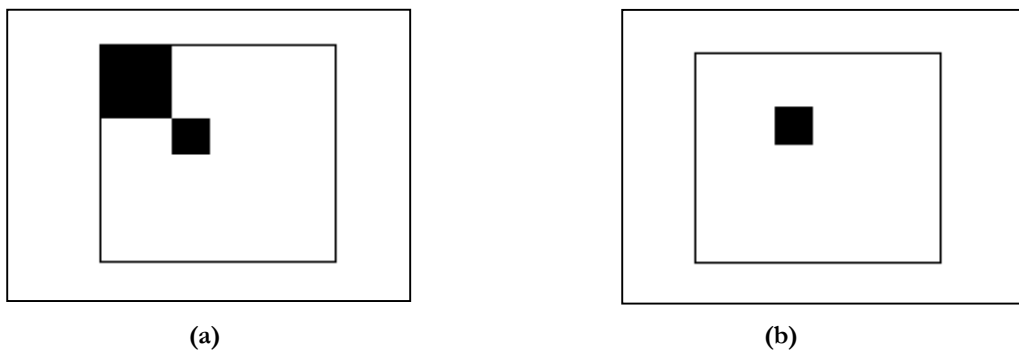


Fig.9: (a) Imagen sintética en el instante t ; (b) Imagen sintética en el instante $t+1$.

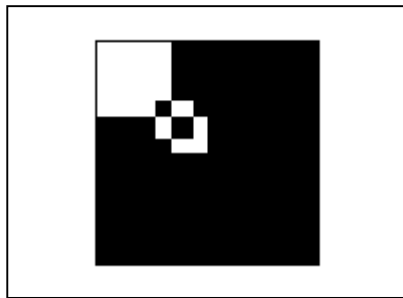


Fig.10 Imagen sintética resultado de sustraer a la imagen en el instante $t+1$, la imagen en el instante inmediatamente anterior.

Los resultados obtenidos con imágenes sintéticas sirvieron de apoyo para el tratamiento de imágenes reales, empleando así la misma metodología.

Para los primeros fotogramas y en uno de cada diez se detectará movimiento en toda la imagen. Para el resto de las iteraciones, lo haremos sobre la región de interés.

Para seguir objetos en secuencias donde el fondo no es controlado se han utilizado técnicas de detección de movimiento.

En algunos videos de prueba en los que el movimiento de los objetos es muy lento, la región a localizar es apenas visible. Para aumentar las dimensiones de esta región, la imagen diferencia se calcula como: $I_d = |I_t - I_{t-1}| + |I_{t-1} - I_t|$, donde I_t y I_{t-1} se corresponden con las intensidades en el instante actual y anterior, respectivamente. Para que esta imagen diferencia proporcione una región homogénea y uniforme, aplicamos la operación morfológica del cierre de la imagen.



Fig.11 Imagen resultado de la suma de las diferencias de dos imágenes consecutivas de una secuencia de vídeo, y del cierre morfológico de la misma.

6.2. Pruebas y resultados de segmentación y determinación de la posición.

Se han realizado cálculos de regiones de interés en imágenes sintéticas, utilizando la técnica de *conectividad de regiones*.

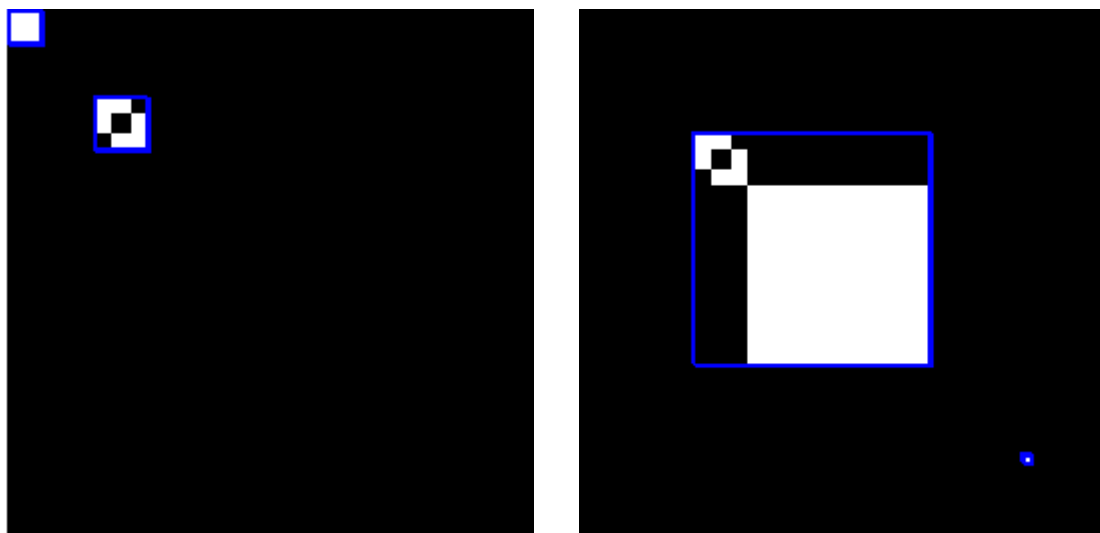


Fig.12 Ejemplos de segmentación por conectividad con imágenes sintéticas.

En el tratamiento de secuencias de imágenes reales hemos implementado segmentación basada en conectividad y en crecimiento de regiones.

Las imágenes no están exentas de ruido en forma de píxeles independientes. El uso de 8-conectividad reduce la posibilidad de confundir uno de estos píxeles como un objeto en movimiento.

En la segmentación por crecimiento de regiones se establece que un píxel forma parte de un objeto si el valor de intensidad de éste supera cierto umbral. Una región de interés que encuadre el objeto segmentado está definida por cuatro vértices. El primero de ellos se corresponde con el primer píxel que supere el umbral. Los restantes vértices se escogen a partir de él aumentando una distancia constante para formar una región cuadrada. El tamaño de la ventana aumenta progresivamente hasta que se engloba a todo el objeto.

Podemos observar que la implementación apoyada en el método de segmentación basado en conectividad resulta más eficiente.

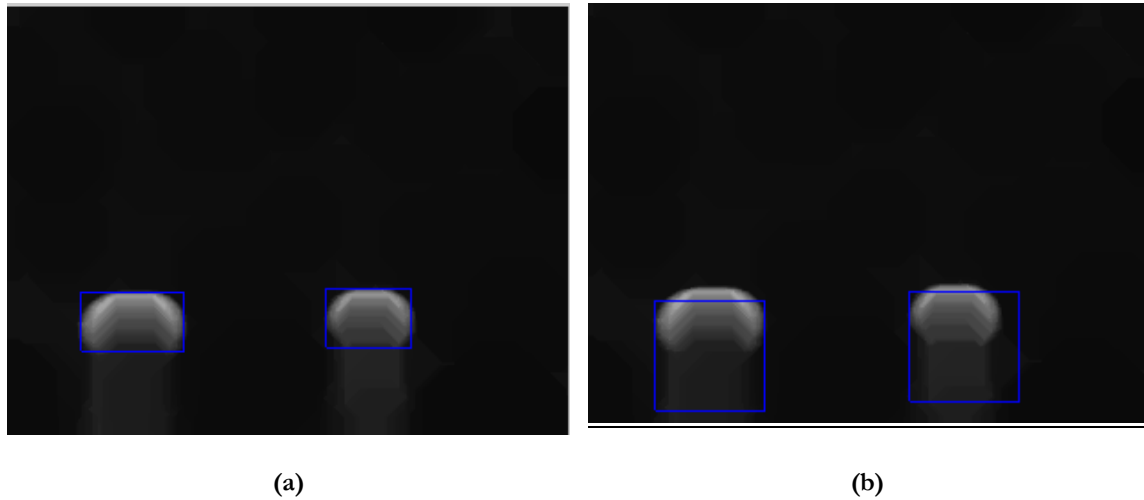


Fig13: (a) Segmentación por conectividad; (b) segmentación por crecimiento de regiones

El centro de masas del objeto en movimiento se busca en la región de interés de la imagen diferencia. En la Figura 14 mostramos este hecho y representamos el centro de masas con un punto de color rojo.

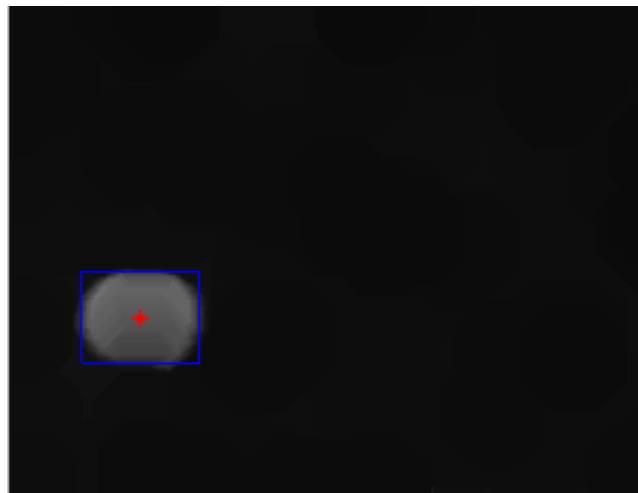


Fig.14 Ejemplo de detección de movimiento, segmentación y determinación de la posición del centro de masas de un objeto.

6.3. Pruebas y resultados en el seguimiento de objetos.

Para estudiar el comportamiento del filtro se realizaron distintas pruebas de seguimiento bajo diferentes condiciones experimentales.

6.3.1. Seguimiento de un objeto sobre fondo negro.

Presentamos los resultados del seguimiento de un objeto bajo diferentes condiciones iniciales de P_k^- y R_k . La secuencia de imágenes representa el desplazamiento de una pelota de golf de color naranja con movimiento plano paralelo a la imagen.

El centro de masas del objeto viene representado por un punto negro cuyas coordenadas se definen como el parámetro de entrada del filtro u observación y la ventana de color azul muestra la región de interés que se procesará en la siguiente imagen. Las dimensiones de la ventana se establecen en función de la estimación y su incertidumbre.

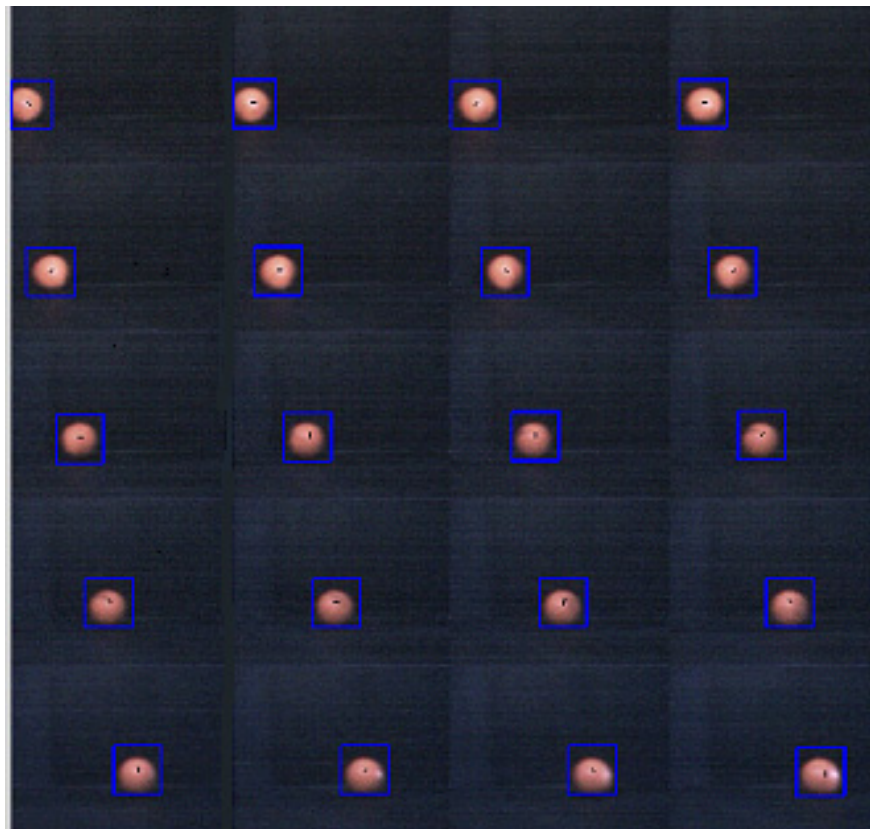


Fig.15. Ejemplo del seguimiento de un objeto sobre un fondo negro, con valores $P_k^-=100$, $R_k=0.02$ y $Q=10^{-3}$

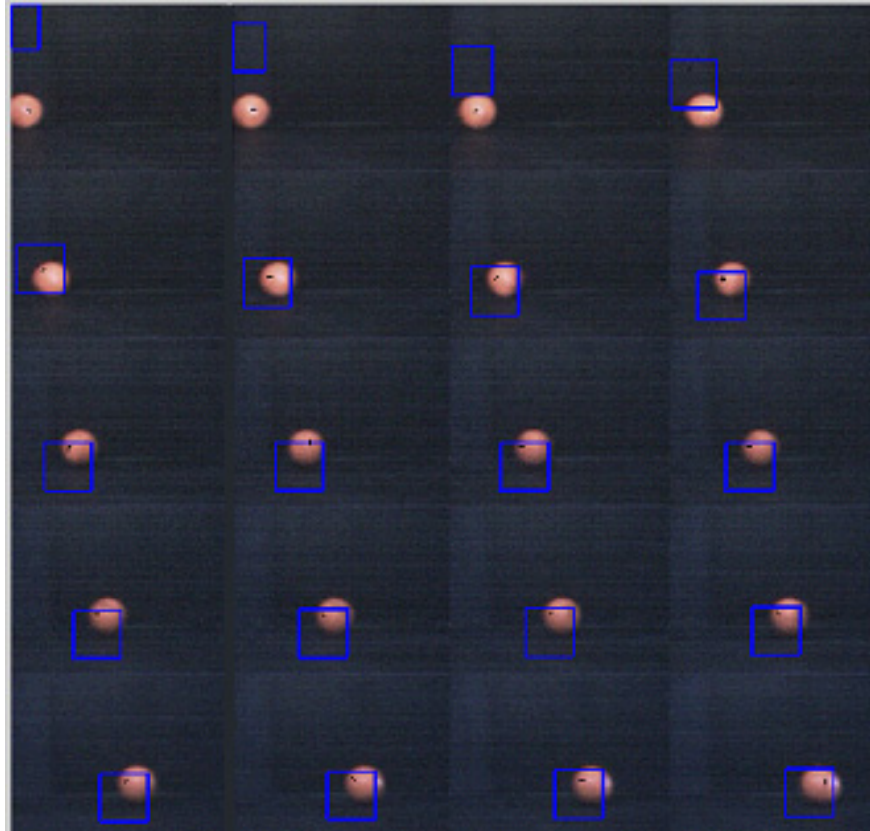


Fig.16. Ejemplo del seguimiento de un objeto sobre un fondo negro, con valores $P_k=0.1$, $R_k=1$ y $Q=10^{-3}$

Podemos observar que la covarianza del error de la medida R_k , y el estimador de la covarianza del error a priori P_k^- , influyen en la evolución del filtro. Cuando aumentamos la confianza en la medida, el sistema encuentra rápidamente al objeto y no lo pierde en ningún momento a lo largo de la secuencia. Cuando confiamos más en el filtro que en la observación, el seguimiento es menos eficiente, se tarda más en localizar al objeto. Además, una vez localizado el sistema no es capaz de encuadrarlo correctamente hasta pasado un determinado número de fotogramas. Esto es debido a que tardamos en corregir la tendencia del estimador.

Los resultados se muestran sobre la imagen original, aunque el procesamiento se realice sobre la imagen diferencia. Sin embargo, puede resultar interesante mostrar los resultados del seguimiento sobre la secuencia de imágenes diferencia. (Fig.17). El centro de masas del objeto se representa mediante un punto rojo para mejorar la visualización.

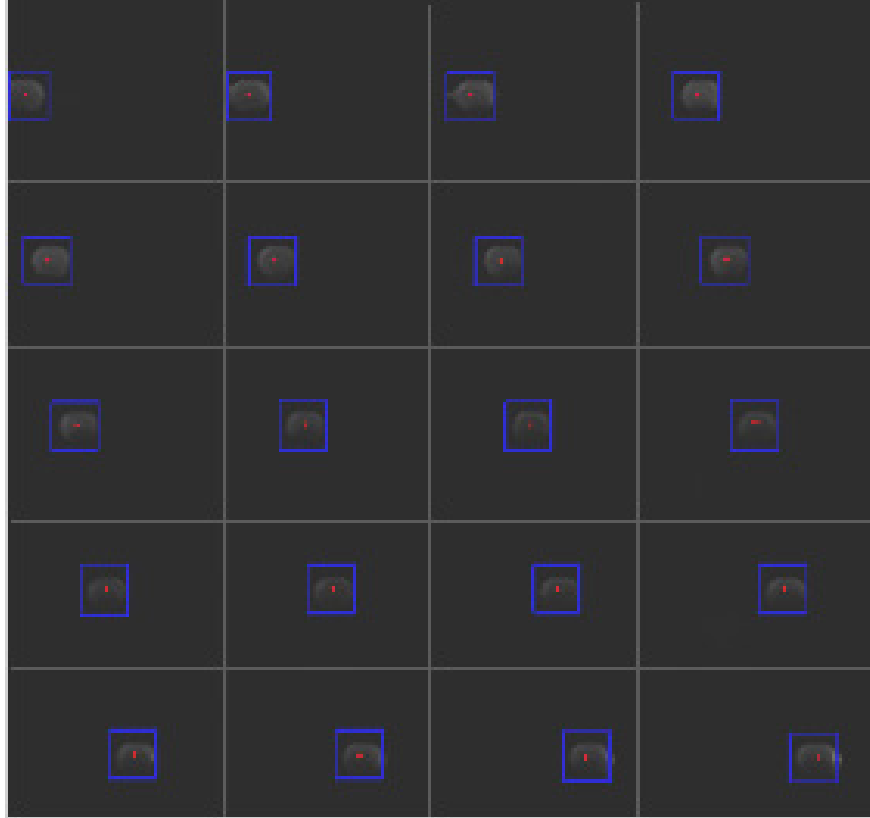


Fig.17. Ejemplo del seguimiento de un objeto sobre un fondo negro, con valores $P_k=100$, $R_k=0.02$ y $Q=10^{-3}$, mostrado sobre la imagen diferencia.

6.3.2. Seguimiento de un objeto en fondo no controlado.

La secuencia de imágenes muestra una persona que se desplaza con un movimiento continuo.(Fig18).

En este caso, el centro de masas del objeto se representa por un punto de color rojo.



Fig.18. Ejemplo del seguimiento de un objeto sobre un fondo no controlado, con valores $P_k=100$, $R_k=0.2$ y $Q=10^{-3}$

En este ejemplo podemos ver cómo en algún fotograma intermedio, la ventana crece. Esto es debido a que el móvil se desplaza tan lentamente que no se detecta movimiento. Esta situación se resuelve aumentando la región de interés y devolviendo como observación la última estimación.

En el seguimiento de un objeto que se mueve con suficiente rapidez, la ventana de interés permanece constante. Podemos afirmar que la detección es más eficiente en este caso. Además, el ejemplo que mostramos en la Fig.19, la ventana de interés tiene mayores dimensiones, con lo que es más fácil detectar movimiento.

El centro de masas del objeto se representa por un punto de color amarillo.

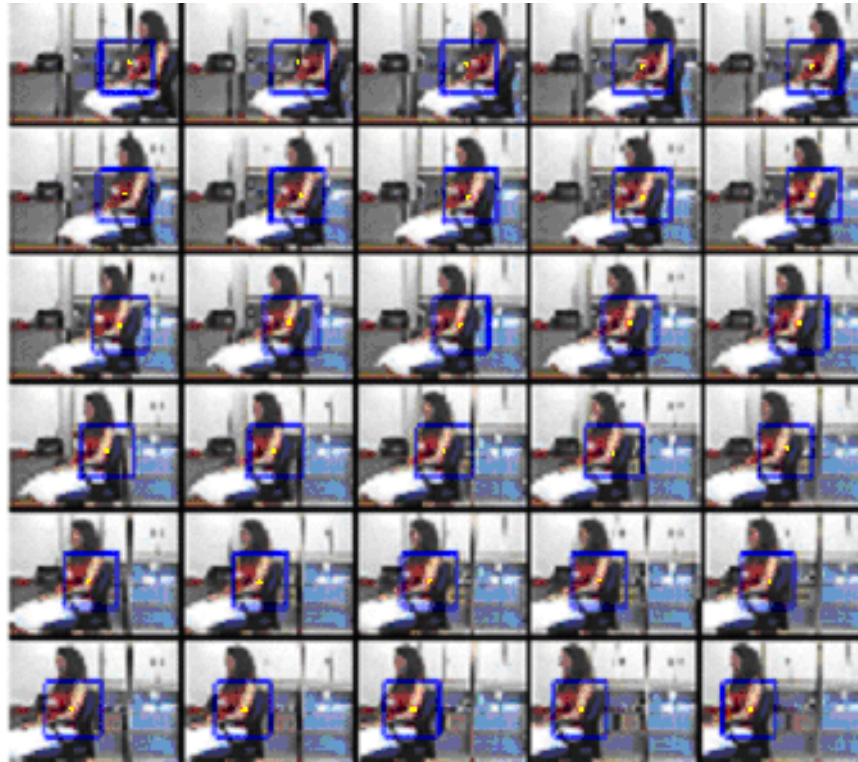


Fig.19. Ejemplo del seguimiento de un objeto sobre un fondo no controlado, con valores $P_k=100$, $R_k=0.2$ y $Q=10^{-3}$

6.3.3. Seguimiento de varios objetos sobre fondo negro.

El filtro de Kalman modela distribuciones unimodales, lo que implica que sólo puede seguir un objeto. Para seguir varios objetos es necesario proporcionar un filtro a cada uno. Estos filtros evolucionan de manera independiente. Se realizaron experimentos para evaluar el comportamiento del filtro en esta situación. Una de estas pruebas la recogemos en la Fig.20.

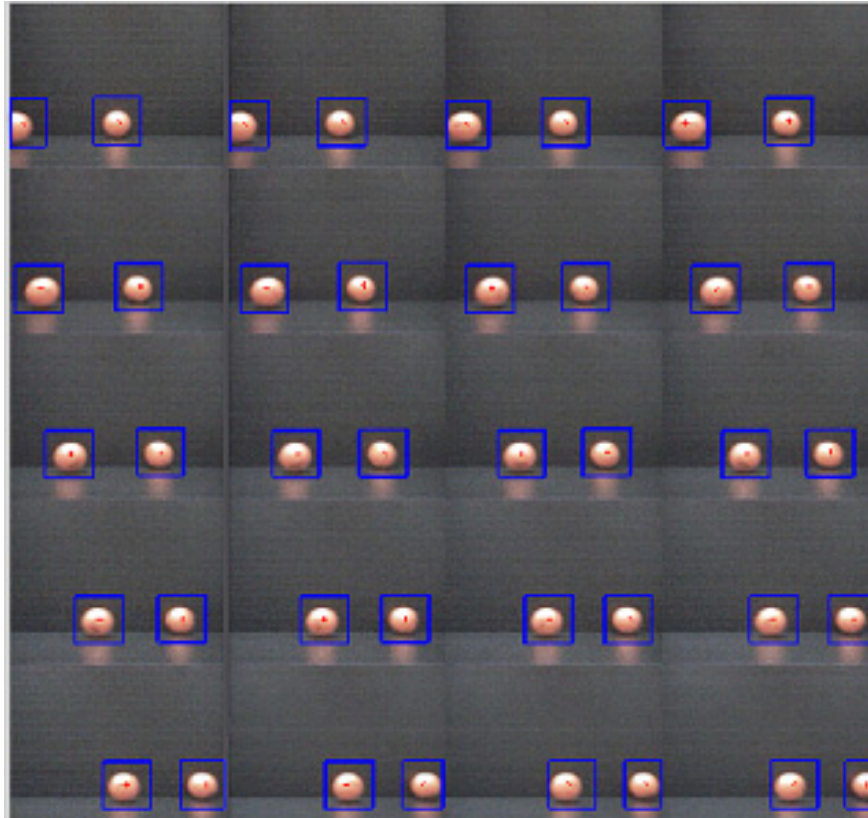


Fig.20. Seguimiento de dos objetos sobre fondo negro.

Cuando uno de los objetos desaparece de la escena el sistema se comporta como si se tratara de un caso de oclusión, aumentando la ventana que lo encuadraba. Cuando se procese toda la imagen de nuevo y se detecte sólo un objeto el sistema mantendrá un único filtro activo.

Si lo que ocurre es que aparece un nuevo objeto en la escena, éste no será detectado hasta que no se procese toda la imagen.

6.3.4. Seguimiento de varios objetos en fondo no controlado.

En la Fig.21 se muestra una secuencia de imágenes donde aparecen tres personas en movimiento. Podemos apreciar además que cada persona se mueve en un plano de profundidad distinto.

Para este caso se lanzan tres filtros, uno para cada persona, y cada uno evoluciona de manera independiente.



Fig.21 Seguimiento de tres personas en fondo no controlado.

Para el correcto seguimiento de varios objetos sobre un escenario real es imprescindible detectar los movimientos y segmentar la imagen. De lo contrario el sistema no es capaz de determinar el número de objetos presentes en la imagen, imposibilitando su correcto seguimiento.

Mostramos en este caso una secuencia de imágenes donde dos personas caminan en sentidos opuestos.

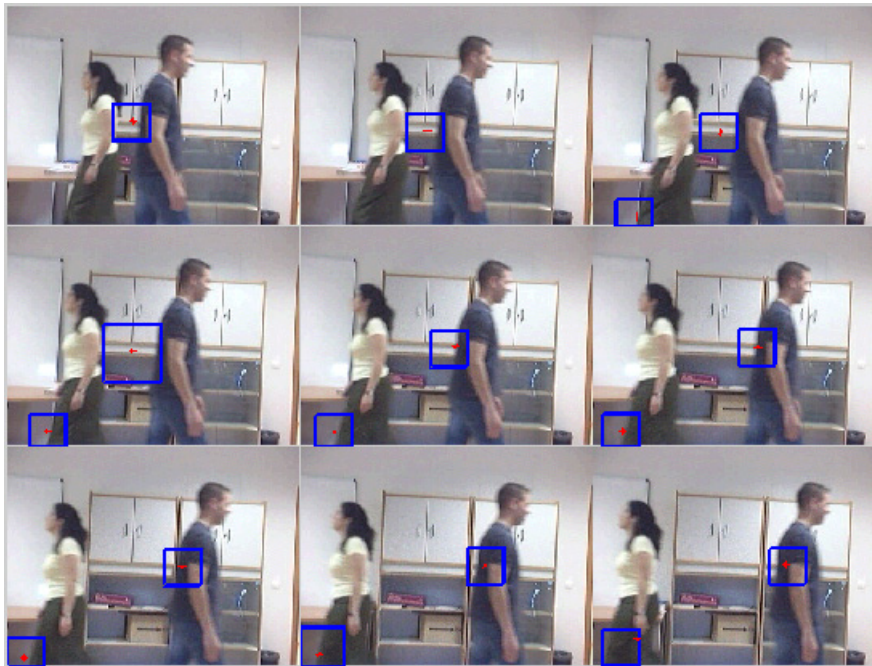


Fig.22 Seguimiento de dos personas en fondo no controlado.

En los dos primeros fotogramas sólo vemos una ventana de interés porque las figuras están muy próximas. A partir del tercer fotograma el seguimiento se realiza para dos figuras, gracias a que coincide con el procesamiento de toda la imagen. Nótese además que en el cuarto fotograma una de las ventanas crece porque no detecta movimiento, y es en el fotograma siguiente cuando recupera al objeto.

El sentido del movimiento de los objetos no plantea ningún tipo de problema en el seguimiento.

6.3.5. Seguimiento de un objeto bajo condiciones de oclusión.

Por oclusión nos referimos a la situación en la que el objeto móvil desaparece durante un cierto número de frames para, transcurrido este tiempo, reaparecer en la imagen.

En caso de oclusión el seguimiento también evolucionará de distinta manera, dependiendo de las condiciones iniciales que proporcionamos al filtro.

A continuación mostramos un ejemplo en el que una pelota de golf se mueve sobre un fondo negro y en un determinado momento se oculta. Para este caso los parámetros de error fueron: $P_k=10$, $R_k=0.2$ y $Q=10^{-3}$.

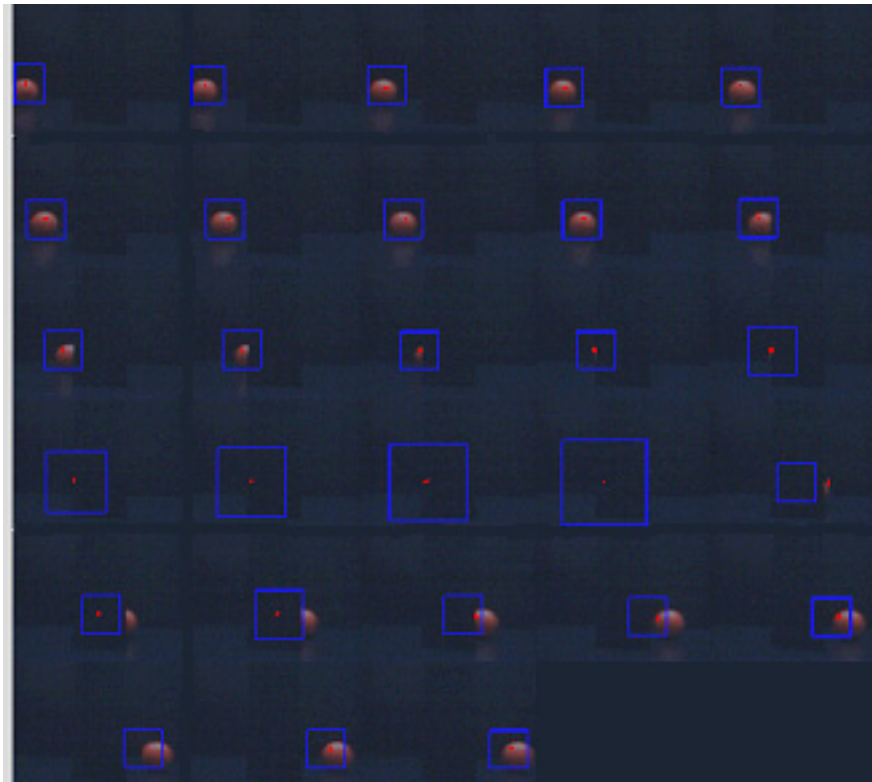


Fig.23. Secuencia de imágenes con oclusión. $P_k=10$, $R_k= 0.2$ y $Q=10^{-3}$.

Cuando el objeto desaparece y confiamos mucho en la observación, como en este caso, la ventana de interés permanece prácticamente inmóvil, aumentando además sus dimensiones. Cuando el objeto reaparece y la región de interés recoge de nuevo al objeto, el seguimiento se retoma de manera normal. Esta situación requiere que el procesamiento de toda la imagen se realice cada determinado número de frames o que la ventana crezca suficientemente. De esta forma se garantiza que no transcurre demasiado tiempo en encontrar de nuevo al objeto cuando este reaparece.

La figura 24 ilustra el mismo ejemplo que la figura 23 salvo que presenta distintos valores de error: $P_k=0.1$, $R_k=0.7$ y $Q=10^{-3}$.

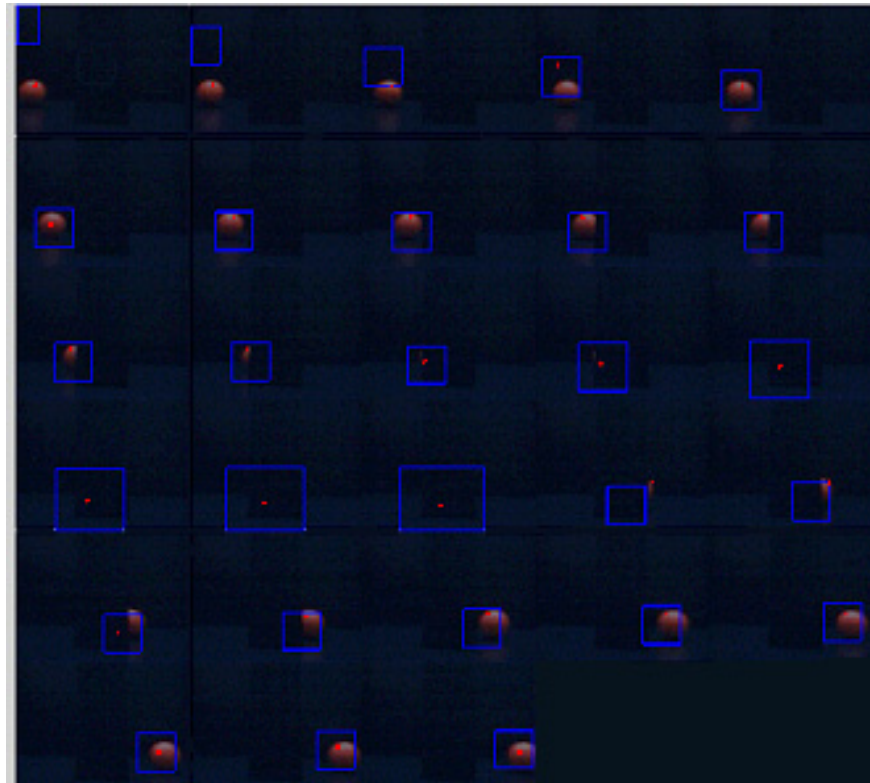


Fig.24. Secuencia de imágenes con oclusión. $P_k=0.1$, $R_k=0.7$ y $Q=10^{-3}$.

Cuando confiamos en el estimador y se produce una oclusión, la ventana de interés continúa con el movimiento que éste llevaba, aunque algo ralentizado. Y al igual que en el ejemplo anterior la ventana crece, aumentando así el espacio de búsqueda. Pero al reaparecer el objeto y obtener la nueva observación, como recae más confianza sobre el filtro que sobre la observación, se tarda más en encuadrar correctamente al objeto.

Diremos entonces que el rendimiento óptimo del filtro se alcanzará cuando se consiga un compromiso entre los valores de R_k y P_k .

La siguiente figura muestra un ejemplo de oclusión en escenario real.



Fig.25. Secuencia de imágenes con oclusión. $P_k=100$, $R_k=0.02$ y $Q=10^{-3}$.

Como en este ejemplo confiamos mucho en la observación, el filtro se recupera rápidamente al reaparecer el objeto.

6.3.6. Seguimiento de un objeto en tiempo real.

Ambos ejemplos muestran el seguimiento de un objeto en la aplicación implementada en tiempo real. El centro de masas se representa con un punto rojo, la posición estimada se representa con un punto verde y la región de interés mediante un cuadrado azul celeste.



Fig.26 Seguimiento de una persona en tiempo real.



Fig.27 Seguimiento de un objeto en tiempo real.

Otra forma de ver los resultados es mediante el uso de gráficos. Véanse a continuación las gráficas obtenidas para los ejemplos expuestos en las Figuras 15 y 16.

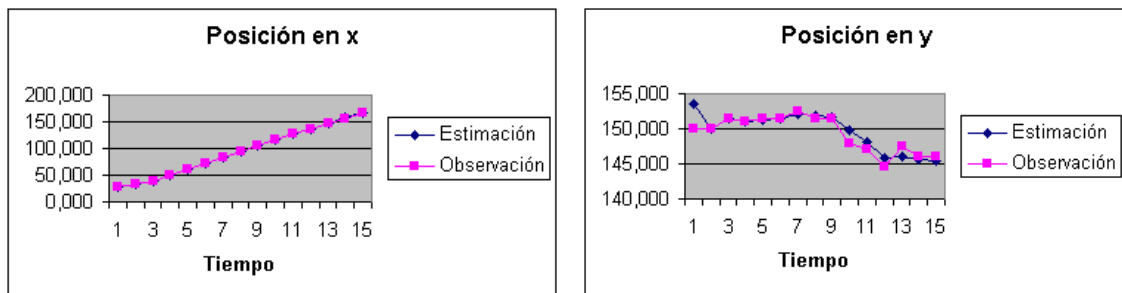


Fig.28 Gráficos de la posición observada y estimada para los valores $P_k=100$ y $R_k=0.02$.

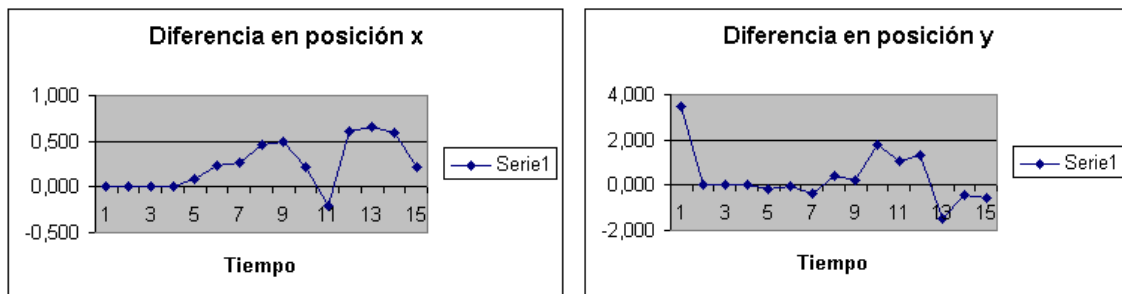


Fig.29 Gráficos de la diferencia en la posición para los valores $P_k=100$ y $R_k=0.02$.

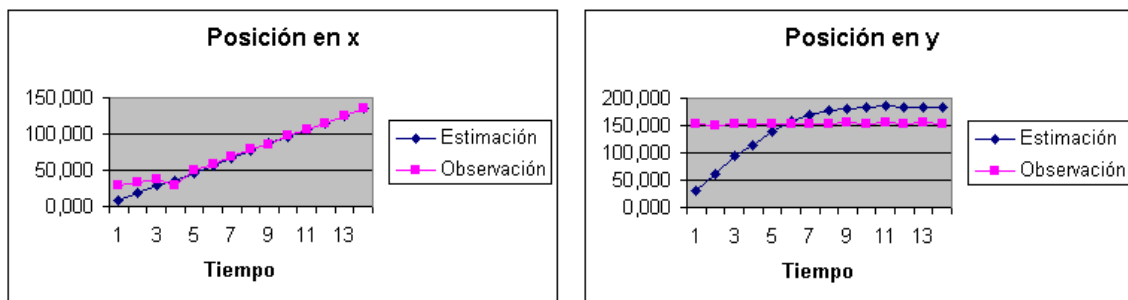


Fig.30 Gráficos de la posición observada y estimada para los valores $P_k=0.1$ y $R_k=1$

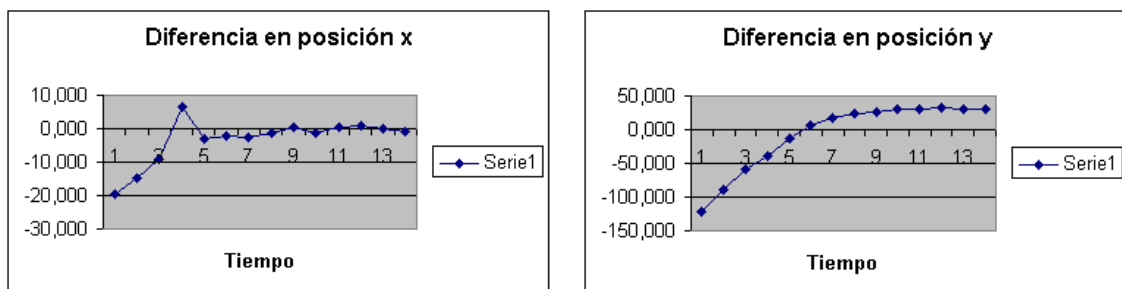


Fig.31 Gráficos de la diferencia en la posición para los valores $P_k=0.1$ y $R_k=1$

VI. CONCLUSIONES Y AVANCES FUTUROS

En este trabajo se ha presentado una implementación del filtro de Kalman aplicado al seguimiento de objetos en secuencias de imágenes.

La línea de investigación dentro de la que se enmarca este proyecto, ofrece un camino directo para abordar el problema general del seguimiento de objetos. De aquí se pueden extraer varias conclusiones importantes.

Una primera conclusión recae en la importancia de seleccionar una buena técnica de segmentación de los objetos de interés, ya que esta tarea es crucial a la hora de realizar un buen seguimiento los mismos. Resulta obvio que si el objeto no se logra segmentar, difícilmente podrá seguirse.

Otra conclusión a tener en cuenta, y es aquí donde reside la importancia del filtro, es la necesidad de reducir al máximo los espacios de búsqueda en la escena, de manera que el coste computacional del seguimiento de los objetos disminuya proporcionalmente.

Hemos abordado con éxito los problemas de detección de movimiento y segmentación de objetos en secuencias de imágenes sintéticas, para posteriormente aplicar esta misma metodología al tratamiento de imágenes reales.

Otro de los propósitos alcanzados reside en el comportamiento óptimo del filtro bajo distintas condiciones experimentales. Logramos solventar problemas de oclusión, así como realizar el seguimiento de varios objetos.

En definitiva, podemos afirmar que el filtro de Kalman constituye un buen método para hacer predicciones a lo largo del tiempo de cualquier suceso evolutivo. Este suceso puede ser el movimiento de un objeto, el caudal que baja por un río en régimen de avenida, la demanda de material de un almacén, etc.

A pesar de los logros obtenidos en la realización de este proyecto, quedan muchos caminos por explorar en este campo y probablemente algunas cosas que mejorar. Y puesto que se ha empleado una metodología de programación estructurada y modular, es muy sencillo modificar o extender las prestaciones de la aplicación. A continuación se exponen brevemente algunas de las líneas en las que sería interesante profundizar para dar continuidad a este proyecto.

Como ya sabemos, se planteó desarrollar la aplicación en tiempo real, lo que fue posible realizar y por ello hemos incluido como parte importante del proyecto. Pero esta línea de desarrollo deja pendiente que la aplicación sea capaz de seguir varios objetos de manera simultánea, de manera que se proporciona una vía abierta donde seguir investigando.

Otro estudio de interés sería desarrollar todo el proceso de seguimiento con el uso de cámaras móviles mediante actuadores motorizados. Un ejemplo de utilización de esta técnica que se plantea reside en el uso de cámaras de vigilancia, o dispositivos de seguridad, donde los movimientos propios de la cámara pueden estar restringidos y controlados.

También planteamos como línea de futuro trabajo la implementación de una interfaz gráfica que, añadida a nuestra aplicación, serviría de apoyo didáctico al estudio del filtro de Kalman.

VII. BIBLIOGRAFÍA

[1] Gonzalo Pajares. Jesús M. De la Cruz. **“Visión por Computador. Imágenes digitales y aplicaciones”**. Ed. Ra-Ma. 2001.

[2] Javier González Jiménez. **“Visión por Computador”**. Ed. Paraninfo. 2000.

[3] José Francisco Vélez. Ana Belén Moreno. Ángel Sánchez. José Luis Esteban. **“Visión por Computador”**. Ed. Universidad Rey Juan Carlos. 2003.

[4] Donald Gillies.Yuxin Zheng, **“Dynamic Interactions with the Philosophy of Mathematics, Theoria”**, 16, pp. 437-459. 2001.

[5] Kalman, R.E., **“A new approach to linear filtering and prediction problems”**, Trans. ASME, Journal of Basic Engineering, pp.34-45, 1960.

[6] R. Sargent. B. Bailey. C.Witty. A. Wrioth. **“Dynamic object capture using fast vision tracking”**, Artificial Intelligence Magazine, vol.18:1, pp.65-72,1997.

[7] R. Isermann. **“Digital control systems”**. Ed. Springer-Verlag.1991.

[8] J. Meditch. **“Stochastic Optimal Linear Estimation and Control”**. Ed. McGraw Hill. 1969.